Source code

26 files

app.py

```
"""Hugging Face Space: Certified AC robust secure sets with GENCO in the loop.

Tabs
  1. Results     - the 24-hour study (DC Benders vs big-M MILP, AC-corrected RSS,
                   GENCO screening and adversary), read from results/*.csv
  2. Live DC-RSS - solve the RSS bilevel for any hour with the two-phase
                   Benders decomposition (about 1 s) and see the certificate
  3. GENCO probe - GENCO's internal power-mismatch trajectory r^(1..K) for
                   seen and unseen topologies; physics-trajectory recalibration
  4. Conformal   - fit the screen live for any alpha / method and see which
                   scenarios are cleared without exact AC (guarantee P(miss) <= alpha)
All numbers shown are produced by the code in genco_rss/ (no constants).
"""
import json
import os

import gradio as gr
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from genco_rss import benders, rss
from genco_rss.grid import Grid

HERE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(HERE, "results")
C = {"blue": "#2a78d6", "orange": "#eb6834", "aqua": "#1baf7a", "yellow": "#eda100", "gray": "#8a8984"}
GRID = Grid.load()


def _read(name):
    p = os.path.join(RES, name)
    return pd.read_csv(p) if os.path.exists(p) else None


def _style(ax):
    ax.grid(alpha=0.25, lw=0.6)
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)


# ---------------------------------------------------------------------- tab 1
def results_view():
    e1, e4 = _read("e1_dc_rss.csv"), _read("e4_ac_rss.csv")
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.2), dpi=120)
    if e1 is not None:
        up = e1[e1.limit == "upper"].set_index("hour"); lo = e1[e1.limit == "lower"].set_index("hour")
        ax[0].fill_between(up.index, lo.alpha, up.alpha, color=C["blue"], alpha=0.18, label="DC robust secure
range (Benders)")
        ax[0].plot(up.index, up.alpha, color=C["blue"], lw=2)
        ax[0].plot(lo.index, lo.alpha, color=C["blue"], lw=2)
        if e4 is not None:
            u4 = e4[e4.limit == "upper"].set_index("hour"); l4 = e4[e4.limit == "lower"].set_index("hour")
            ax[0].plot(u4.index, u4.alpha_ac, color=C["orange"], lw=2, ls="--", label="AC-corrected limits")
            ax[0].plot(l4.index, l4.alpha_ac, color=C["orange"], lw=2, ls="--")
        ax[0].set_xlabel("hour"); ax[0].set_ylabel("RSS index  (sum of non-slack outputs, p.u.)")
        ax[0].legend(frameon=False, fontsize=8); _style(ax[0])
        ax[1].bar(np.arange(2) - 0.0, [e1.milp_s.mean(), e1.benders_s.mean()], color=[C["gray"], C["blue"]],
width=0.5)
        ax[1].set_xticks([0, 1], ["big-M KKT MILP\n(RSS_MILP2)", "two-phase Benders"])
        ax[1].set_ylabel("mean seconds per bilevel solve (measured)"); _style(ax[1])
    fig.tight_layout()
```

```
    md = ""
    npath = os.path.join(RES, "numbers.json")
    if os.path.exists(npath):
        N = json.load(open(npath))
        g = lambda k, f="{:.0f}": f.format(N[k]) if k in N else "n/a"
        md = (f"**Certified DC-RSS (E1).** {g('benders_certified')} of {g('e1_problems')} bilevel problems
certified optimal; "
              f"Benders {g('benders_mean_s', '{:.2f}')} s vs big-M MILP {g('milp_mean_s', '{:.2f}')} s on
average. "
              f"The big-M MILP misreports {g('bigm_err_n')} problems (up to {g('bigm_err_max_MW', '{:.2f}')}
MW).\n\n"
              f"**AC correction (E4).** {g('it1_dc_boundary_insecure')} of {g('it1_n')} DC limits are AC-
insecure; "
              f"mean secure range {g('e4_width_dc_mean_MW')} MW (DC) -> {g('e4_width_ac_mean_MW')} MW (AC-
corrected); "
              f"empty in hour(s) {N.get('e4_empty_hour_list', '-') or '-'}. PTR cleared
{g('pool_cleared_pct')} % of "
              f"{g('pool_total')} pool scenarios; audit found {g('missed_total')} missed insecure
scenario(s).\n\n"
              f"**Screening (E3, alpha = 0.05).** PTR clears {g('clear_ptr_005', '{:.1f}')} % of secure
scenarios "
              f"(plain conformal {g('clear_plain_005', '{:.1f}')} %), observed miss {g('miss_ptr_005',
'{:.2f}')} %.")
    tables = [t for t in (e1, e4) if t is not None]
    return fig, md, (tables[-1].round(4) if tables else pd.DataFrame())


# ---------------------------------------------------------------------- tab 2
def live_rss(hour, limit):
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(GRID, int(hour))
    d1, d2 = (-d, d) if limit == "upper" else (d, -d)
    r = benders.solve(A, B, b, np.zeros(A.shape[1]), d1, d2, p_lb, p_ub, u_ub)
    sgn = -1 if limit == "upper" else 1
    npv = GRID.npv
    at_ub = int(np.isclose(r.p, p_ub).sum()); at_lb = int(np.isclose(r.p, p_lb).sum())
    md = (f"### Hour {int(hour)}, {limit} limit\n"
          f"* RSS limit alpha = **{sgn * r.obj:.5f} p.u.** (attained by the exact follower reaction)\n"
          f"* certified bound = {sgn * r.bound:.5f} p.u., relative gap {r.gap:.1e}, status `{r.status}`\n"
          f"* Benders iterations {r.iters}, cuts {r.cuts}, phase 2 used: {r.phase2}, time {r.time:.2f} s\n"
          f"* worst-case vertex: {at_ub} PV/load variables at their upper bound, {at_lb} at the lower
bound\n"
          f"* worst-case PV output {r.p[:npv].sum() * 100:.1f} MW, load {r.p[npv:].sum() * 100:.1f} MW")
    fig, ax = plt.subplots(figsize=(7, 3.2), dpi=120)
    ax.bar(np.arange(len(r.u)), r.u * 100, color=C["blue"], width=0.7)
    ax.set_xlabel("dispatchable unit"); ax.set_ylabel("dispatch at the worst case [MW]"); _style(ax)
    fig.tight_layout()
    return md, fig


# ---------------------------------------------------------------------- tab 3
def _e3():
    sp, up = os.path.join(RES, "e3_screen.parquet"), os.path.join(RES, "e3_unseen.parquet")
    if not os.path.exists(sp):
        return None, None
    return pd.read_parquet(sp), (pd.read_parquet(up) if os.path.exists(up) else None)


def probe_view():
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.2), dpi=120)
    md = "Run `python genco_ac_rssr_challenge.py` to produce the GENCO results."
    scr, uns = _e3()
    if scr is not None:
        rcols = sorted([c for c in scr.columns if c[0] == "r" and c[1:].isdigit()], key=lambda c:
int(c[1:]))
        for name, df, col in (("seen topologies", scr, C["blue"]), ("unseen topology 49-66", uns,
C["orange"])):
            if df is None:
                continue
            m = df[rcols].median().values
            ax[0].plot(np.arange(1, len(m) + 1), m, marker="o", ms=4, color=col, lw=2, label=name)
        ax[0].set_yscale("log"); ax[0].set_xlabel("GENCO corrective layer l"); ax[0].set_ylabel("median
mismatch ||r(l)||")
        ax[0].legend(frameon=False); _style(ax[0])
    ab = os.path.join(RES, "e3_ptr_ablation.csv")
    if os.path.exists(ab):
        t = pd.read_csv(ab, index_col=0)
```

```
        y = np.arange(len(t))
        ax[1].barh(y + 0.2, t.test_MAE * 100, height=0.38, color=C["blue"], label="seen topologies")
        ax[1].barh(y - 0.2, t.unseen_topology_MAE * 100, height=0.38, color=C["orange"], label="unseen
topology")
        ax[1].set_yticks(y, t.index, fontsize=8); ax[1].invert_yaxis()
        ax[1].set_xlabel("max-loading MAE [% of rating]"); ax[1].legend(frameon=False); _style(ax[1])
        md = ("**Physics-trajectory recalibration.** GENCO's own per-layer AC mismatch trajectory explains a
large "
              "part of its error. Rows: what the recalibrator g sees (fitted on the tune split only).\n\n"
              + t.round(4).to_markdown())
    fig.tight_layout()
    return fig, md


def screen_view(alpha, method):
    from genco_rss.conformal import Screen
    scr, uns = _e3()
    if scr is None:
        return None, "No E3 results yet."
    perm = np.random.default_rng(0).permutation(len(scr))
    tune, cal, test = scr.iloc[perm[:1000]], scr.iloc[perm[1000:2000]], scr.iloc[perm[2000:]]
    S = Screen(alpha=float(alpha), method=method).fit(tune, cal)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.2), dpi=120)
    rows = []
    for i, (name, d) in enumerate((("test (seen topologies)", test), ("unseen topology 49-66", uns))):
        if d is None:
            continue
        f = S.flag(d)
        ax[i].scatter(d.load_true[~f], S.upper(d, "load")[~f], s=7, color=C["aqua"], label="cleared (no exact
AC)")
        ax[i].scatter(d.load_true[f], S.upper(d, "load")[f], s=7, color=C["gray"], alpha=0.5, label="sent to
exact AC")
        ax[i].axvline(1, color="k", lw=0.8, ls=":"); ax[i].axhline(1, color="k", lw=0.8, ls=":")
        ax[i].set_xlabel("exact AC max loading"); ax[i].set_ylabel("conformal upper bound");
ax[i].set_title(name)
        ax[i].legend(frameon=False, fontsize=8); _style(ax[i])
        r = S.report(d); rows.append({"set": name, **{k: r[k] for k in ("n", "insecure", "flagged",
"missed", "secure_cleared_frac", "miss_rate")}})
    fig.tight_layout()
    md = (f"**alpha = {alpha}, method = {method}.** Guarantee: P(insecure and cleared) <= alpha.\n\n"
          + pd.DataFrame(rows).round(4).to_markdown(index=False))
    return fig, md


with gr.Blocks(title="GENCO certified AC-RSS") as demo:
    gr.Markdown("# Certified AC robust secure sets with GENCO in the loop\n"
                "Two-phase Benders decomposition (certified DC bounds) + GENCO screening with physics-
trajectory "
                "recalibration (conformal guarantee) and gradient adversary + exact AC verification, on PGLib
IEEE-118 with 20 PV buses and the "
                "69-70 / 100-103 contingencies. Built with gridfm-graphkit and gridfm-datakit.")
    with gr.Tab("Results"):
        b1 = gr.Button("Load results")
        f1 = gr.Plot(); m1 = gr.Markdown(); t1 = gr.Dataframe()
        b1.click(results_view, outputs=[f1, m1, t1])
    with gr.Tab("Live DC-RSS (Benders)"):
        h = gr.Slider(1, 24, value=12, step=1, label="hour")
        lim = gr.Radio(["upper", "lower"], value="upper", label="RSS limit")
        b2 = gr.Button("Solve")
        m2 = gr.Markdown(); f2 = gr.Plot()
        b2.click(live_rss, inputs=[h, lim], outputs=[m2, f2])
    with gr.Tab("GENCO probe"):
        b3 = gr.Button("Show")
        f3 = gr.Plot(); m3 = gr.Markdown()
        b3.click(probe_view, outputs=[f3, m3])
    with gr.Tab("Conformal screen (live)"):
        a4 = gr.Slider(0.01, 0.2, value=0.05, step=0.01, label="alpha (allowed miss probability)")
        k4 = gr.Radio(["ptr", "normalised", "plain"], value="ptr", label="method")
        b4 = gr.Button("Fit and screen")
        f4 = gr.Plot(); m4 = gr.Markdown()
        b4.click(screen_view, inputs=[a4, k4], outputs=[f4, m4])

if __name__ == "__main__":
    demo.launch()
```

genco_ac_rssr_challenge.py

```
"""THE GENCO CHALLENGE 2026 - one-command reproduction.

Certified AC robust secure sets with GENCO in the loop
(Benders bounds, GENCO with physics-trajectory recalibration and conformal
screening, GENCO-gradient adversary, exact AC verification).

Author: Muhammad Abdillah, Department of Electrical Engineering,
        Universitas Pertamina, Jakarta, Indonesia

Steps (each writes to results/ and can be run alone with --steps):
  data   gridfm-datakit: 24,000 exact AC power flows of RSS operating points
  train  gridfm-graphkit: GENCO (GNS_heterogeneous) power-flow model
  e1     certified DC-RSS, 24 h x 2 limits: Benders vs big-M KKT MILP
  e3     GENCO accuracy, corrective-layer probe, physics-trajectory recalibration (PTR),
         conformal screening (3 methods x 3 alphas x 20 splits)
  e4     AC-corrected RSS with GENCO screening + gradient adversary + exact AC
  e5     extension: second-life EV batteries as RSS dispatch resources
  fig    figures
  numbers results/numbers.json + paper/numbers.tex (every number quoted anywhere)
  docs   render the post / README from docs/templates with those numbers

Re-run the experiments with the shipped trained GENCO (no training):
  python genco_ac_rssr_challenge.py --steps e1,e3,e4,e5,fig,numbers,docs

Use an official GENCO checkpoint instead:
  python genco_ac_rssr_challenge.py --steps e3,e4,fig \
      --genco-run <folder with best_model_state_dict.pt/last.pt + normalizer_stats.pt> \
      --genco-config <training yaml of that checkpoint>
"""
import argparse
import os
import subprocess
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
WORK = os.environ.get("WORK_DIR", os.path.join(HERE, "work"))


def sh(cmd, env=None):
    print("+", " ".join(cmd), flush=True)
    subprocess.run(cmd, check=True, cwd=HERE, env={**os.environ, **(env or {})})


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--steps", default="data,train,e1,e3,e4,e5,fig,numbers,docs")
    ap.add_argument("--scenarios", type=int, default=24000)
    ap.add_argument("--genco-run", default=None,
                    help="folder with best_model_state_dict.pt + normalizer_stats.pt (default: the run
trained "
                         "by --steps train, else the shipped weights/genco_pf_case118)")
    ap.add_argument("--genco-config", default=os.path.join(HERE, "configs", "genco_pf_case118.yaml"))
    a = ap.parse_args()
    steps = a.steps.split(",")
    if a.genco_run is None:
        shipped = os.path.join(HERE, "weights", "genco_pf_case118")
        a.genco_run = os.path.join(WORK, "mlruns") if ("train" in steps or not os.path.isdir(shipped)) else
shipped
    os.makedirs(os.path.join(HERE, "results"), exist_ok=True)
    env = {"GENCO_RUN": a.genco_run, "GENCO_CONFIG": a.genco_config, "WORK": os.path.join(WORK, "eval"),
           "MLFLOW_ALLOW_FILE_STORE": "true", "PYTHONPATH": HERE}
    data_root = os.path.join(WORK, "data", "train")
    if "data" in steps:
        sh([sys.executable, "-m", "genco_rss.datagen", "--scenarios", str(a.scenarios), "--out", data_root],
env)
    if "train" in steps:
        sh(["gridfm_graphkit", "train", "--config", a.genco_config, "--data_path", data_root,
            "--log_dir", a.genco_run, "--exp_name", "genco_pf_case118"], env)
    for s, script in (("e1", "e1_dc_rss.py"), ("e3", "e3_genco_probe_conformal.py"), ("e4", "e4_ac_rss.py"),
                      ("e5", "e5_bess.py"), ("fig", "e6_figures.py"),
                      ("numbers", "e7_numbers.py"), ("docs", "e8_docs.py")):
```

```
        if s in steps:
            sh([sys.executable, os.path.join("experiments", script)], env)


if __name__ == "__main__":
    main()
```

genco_ac_rssr_slbess.py

```
"""THE GENCO CHALLENGE 2026 - extension: second-life EV batteries in the RSS.

Twenty retired-EV batteries (one per PV bus, 35 % of the bus PV capacity,
4 h, state of health 0.72-0.84) are added to the robust-secure-set bilevel as
dispatch (follower) resources. For every hour both RSS limits are solved with
the two-phase Benders decomposition with and without batteries, and the
worst-case operating points are verified with exact AC power flow
(gridfm-datakit + lightsim2grid) in the base case and both contingencies.

Output: results/e5_bess.csv, results/e5_bess_width.csv
Run:    python genco_ac_rssr_slbess.py
"""
import os
import runpy

HERE = os.path.dirname(os.path.abspath(__file__))
os.environ.setdefault("PYTHONPATH", HERE)
runpy.run_path(os.path.join(HERE, "experiments", "e5_bess.py"), run_name="__main__")
```

genco_rss/__init__.py

```

```

genco_rss/acrss.py

```
"""AC-aware robust secure set: DC Benders bounds + GENCO screening + exact AC.

For hour h and limit L in {upper, lower}:

  repeat
    1. DC-RSS by two-phase Benders with the current derating factors
       -> certified alpha_L(h), worst-case PV/load vertex p*.
    2. Screening pool: p* , random vertices of the PV/load box and
       GENCO-gradient adversarial vertices; for each p the dispatch is the
       DC follower response u(p) (what the RSS assumes the operator does);
       each (p, u(p)) is checked in the 3 topologies.
    3. GENCO predicts the security quantities for the whole pool in one
       batch; physics-trajectory recalibration + split-conformal screening
       (conformal.Screen, method 'ptr') flags scenarios.
    4. Exact AC (gridfm-datakit + lightsim2grid) on the flagged scenarios.
    5. AC-correction cuts for each verified violation:
         thermal (MVA) overload on branch l in state s:
             rate_scale[s, l] <- rate_scale[s, l] * (S_rate / S_AC) * (1 - eps)
         slack over capacity (AC losses): slack upper bound reduced.
  until no verified violation (or max_iter).

The final alpha is certified for the corrected DC model (Benders bound) and
every scenario in the pool is AC-secure: flagged ones verified exactly, the
others with the conformal miss guarantee. Voltage and reactive limits are
screened and reported but are not corrected through active-power cuts.
"""
from __future__ import annotations

import os
import shutil
import tempfile
```

```python
import time

import numpy as np
import pandas as pd

from . import benders, datagen, rss
from .grid import Grid
from .conformal import V_TOL

N_STATES = 3
MIN_SCALE = 0.80          # derating floor per AC cut


def follower(A, B, b, d1, d2, p):
    F, u, ok = benders.follower_response(A, B, b, np.zeros(A.shape[1]), d1, d2, p)
    return u if ok else None


def exact_ac(grid: Grid, items, workdir=None):
    """Exact AC power flow (gridfm-datakit + lightsim2grid) for (hour, state,
    p, u) items. Returns (scenario table, per-branch loading [n, n_branch],
    root usable by Genco.evaluate, seconds of AC solver time)."""
    workdir = workdir or tempfile.mkdtemp(prefix="acrss_")
    raw = datagen.generate(grid, datagen.ScenarioList(items), len(items), workdir)
    return read_ac(raw)


def read_ac(raw: str):
    """Security quantities of a gridfm-datakit raw folder (exact AC)."""
    br = pd.read_parquet(os.path.join(raw, "branch_data.parquet"))
    bus = pd.read_parquet(os.path.join(raw, "bus_data.parquet"))
    gen = pd.read_parquet(os.path.join(raw, "gen_data.parquet"))
    rt = pd.read_parquet(os.path.join(raw, "runtime_data.parquet"))
    br["load"] = np.maximum(np.hypot(br.pf, br.qf), np.hypot(br.pt, br.qt)) / br.rate_a
    L = br.pivot_table(index="scenario", columns="idx", values="load").sort_index()
    pq = bus[bus.PQ == 1]
    gen["qex"] = np.maximum(0, np.maximum(gen.q_mvar - gen.max_q_mvar, gen.min_q_mvar - gen.q_mvar))
    sl = gen[gen.is_slack_gen == 1].groupby("scenario")
    tab = pd.DataFrame({
        "load_true": L.max(1),
        "vlow_true": (pq.min_vm_pu - pq.Vm).groupby(pq.scenario).max(),
        "vhigh_true": (pq.Vm - pq.max_vm_pu).groupby(pq.scenario).max(),
        "qex_true": gen.groupby("scenario").qex.max(),
        "slack_p": sl.p_mw.sum(), "slack_pmax": sl.max_p_mw.sum(),
    })
    meta = np.load(os.path.join(raw, "meta.npz"))
    tab["hour"] = meta["hour"][tab.index]
    tab["state"] = meta["state"][tab.index]
    return tab, L, os.path.dirname(os.path.dirname(raw)), float(rt.ac.sum())


def pool_points(grid, A, B, b, d1, d2, p_lb, p_ub, p_star, n_random, rng):
    pts = [p_star]
    for _ in range(n_random):
        pts.append(np.where(rng.random(len(p_lb)) < 0.5, p_lb, p_ub))
    out = []
    for p in pts:
        u = follower(A, B, b, d1, d2, p)
        if u is not None:
            out.append((p, u))
    return out


def ac_rss(grid: Grid, hour: int, limit: str, genco=None, screen=None, n_random=60, n_adv=4,
           adv_steps=6, max_iter=6, eps=0.01, audit=True, log=print):
    """Returns a dict with the DC and AC-corrected RSS limit and the iteration log."""
    rng = np.random.default_rng(hour * 10 + (limit == "upper"))
    nl = len(grid.rate)
    rate_scale = np.ones((N_STATES, nl))
    slack_cut = 0.0
    hist = []
    alpha_dc = None
    for it in range(1, max_iter + 1):
        A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(grid, hour, rate_scale)
        u_ub = u_ub.copy(); u_ub[grid.slack_gen] -= slack_cut
        b = b.copy(); b[-len(u_ub):] = u_ub
```

```
        d1, d2 = (-d, d) if limit == "upper" else (d, -d)
        t0 = time.time()
        res = benders.solve(A, B, b, np.zeros(A.shape[1]), d1, d2, p_lb, p_ub, u_ub)
        t_bend = time.time() - t0
        if res.p is None:          # corrected DC model admits no robust secure dispatch
            log(f"  h{hour:02d} {limit:5s} it{it}: Benders status {res.status} - no robust secure dispatch")
            hist.append({"hour": hour, "limit": limit, "iter": it, "alpha": np.nan, "benders_status":
res.status})
            alpha = np.nan
            break
        alpha = -res.obj if limit == "upper" else res.obj
        if alpha_dc is None:
            alpha_dc = alpha
        pts = pool_points(grid, A, B, b, d1, d2, p_lb, p_ub, res.p, n_random, rng)
        items = [(hour, s, p, u) for (p, u) in pts for s in range(N_STATES)]
        # --- GENCO screening (+ adversarial vertices from the riskiest points)
        t_g = 0.0; n_adv_added = 0; adv_max = np.nan
        tab, L, root, t_ac = exact_ac(grid, items)
        if genco is not None:
            t0 = time.time()
            pred = genco.evaluate(os.path.join(root, "case118_ieee"))
            t_g = time.time() - t0
            pred = pred.set_index("scenario")
            tab = tab.join(pred[[c for c in pred.columns if c.endswith("_pred") or c.startswith("r") or c ==
"pbe_mean"]])
            flagged = screen.flag(tab) if screen is not None else np.ones(len(tab), bool)
            # adversarial refinement around the highest predicted loadings
            if n_adv > 0:
                ds = genco.dataset(os.path.join(root, "case118_ieee"))
                order = np.argsort(-tab["load_pred"].values)
                seen = set()
                new_items = []
                for sc in tab.index.values[order]:
                    k = int(sc) // N_STATES
                    if k in seen:
                        continue
                    seen.add(k)
                    p0 = pts[k][0]
                    idx = [k * N_STATES + s for s in range(N_STATES)]
                    p_adv, _ = genco.adversarial(ds, idx, grid, p0, p_lb, p_ub, steps=adv_steps)
                    p_adv = np.where(p_adv > 0.5 * (p_lb + p_ub), p_ub, p_lb)          # back to a vertex
                    u = follower(A, B, b, d1, d2, p_adv)
                    if u is not None:
                        new_items += [(hour, s, p_adv, u) for s in range(N_STATES)]
                    if len(seen) >= n_adv:
                        break
                if new_items:
                    tab2, L2, root2, t_ac2 = exact_ac(grid, new_items)
                    pred2 = genco.evaluate(os.path.join(root2, "case118_ieee")).set_index("scenario")
                    tab2 = tab2.join(pred2[[c for c in pred2.columns if c.endswith("_pred") or
c.startswith("r") or c == "pbe_mean"]])
                    tab2.index = tab2.index + len(items); L2.index = L2.index + len(items)
                    adv_max = float(tab2.load_true.max())
                    tab = pd.concat([tab, tab2]); L = pd.concat([L, L2]); t_ac += t_ac2
                    flagged = screen.flag(tab) if screen is not None else np.ones(len(tab), bool)
                    n_adv_added = len(new_items) // N_STATES
                    shutil.rmtree(root2, ignore_errors=True)
        else:
            flagged = np.ones(len(tab), bool)
        tab["flagged"] = flagged
        # --- verified violations (exact AC on flagged; audit = all)
        chk = tab[tab.flagged] if not audit else tab
        ver = tab[tab.flagged]
        over = L.loc[ver.index] > 1.0 + 1e-6
        n_line = int(over.values.sum())
        slack_ex = float(np.maximum(0, ver.slack_p - ver.slack_pmax).max()) if len(ver) else 0.0
        vv = int(((ver.vlow_true > V_TOL) | (ver.vhigh_true > V_TOL)).sum())
        missed = int(((~tab.flagged) & (tab.load_true > 1 + 1e-6)).sum()) if audit else -1
        hist.append({"hour": hour, "limit": limit, "iter": it, "alpha": alpha, "bound": -res.bound if limit
== "upper" else res.bound,
                     "benders_status": res.status, "benders_s": t_bend, "pool": len(tab),
"adversarial_points": n_adv_added,
                     "flagged": int(tab.flagged.sum()), "line_violations": n_line, "slack_excess_MW":
slack_ex,
                     "voltage_violations": vv, "missed_by_screen": missed, "max_load_true":
float(tab.load_true.max()),
```

```
                    "max_load_random": float(tab.load_true.iloc[:len(items)].max()), "max_load_adversarial":
adv_max,
                    "max_load_pred": float(tab.load_pred.max()) if "load_pred" in tab else np.nan,
                    "genco_s": t_g, "exact_ac_solver_s": t_ac})
        log(f"  h{hour:02d} {limit:5s} it{it}: alpha={alpha:.4f} pool={len(tab)} flagged=
{int(tab.flagged.sum())} "
            f"line-viol={n_line} slack+={slack_ex:.1f}MW Vviol={vv} maxload={tab.load_true.max():.3f}")
        shutil.rmtree(root, ignore_errors=True)
        if n_line == 0 and slack_ex <= 1e-6:
            break
        # --- AC-correction cuts
        for sc in over.index:
            row = over.loc[sc]
            st = int(tab.loc[sc, "state"])
            for l in row.index[row.values]:
                f = 1.0 / float(L.loc[sc, l])
                rate_scale[st, int(l)] = max(MIN_SCALE, min(rate_scale[st, int(l)], rate_scale[st, int(l)] *
f * (1 - eps)))
        if slack_ex > 0:
            slack_cut += slack_ex / grid.base * (1 + eps)
    return {"hour": hour, "limit": limit, "alpha_dc": alpha_dc, "alpha_ac": alpha, "iters": it,
            "n_derated": int((rate_scale < 1).sum()), "min_scale": float(rate_scale.min()), "history": hist}


def screening_items(grid: Grid, n: int, seed: int = 0, worst=None):
    """i.i.d. draws from the screening distribution used for conformal
    calibration: hour ~ U{1..24}, limit ~ U{upper, lower}, global line derating
    ~ U[0.85, 1], p = Benders worst vertex (prob. 0.2) or a random vertex,
    u = DC follower response, topology ~ U{base, 69-70, 100-103}."""
    rng = np.random.default_rng(seed)
    cache = {}
    items = []
    while len(items) < n:
        h = int(rng.integers(1, 25)); limit = ["upper", "lower"][int(rng.integers(2))]
        k = round(float(rng.uniform(0.85, 1.0)), 2)
        key = (h, k)
        if key not in cache:
            cache[key] = rss.problem(grid, h, np.full((N_STATES, len(grid.rate)), k))
        A, B, b, d, p_lb, p_ub, u_lb, u_ub = cache[key]
        d1, d2 = (-d, d) if limit == "upper" else (d, -d)
        if worst is not None and rng.random() < 0.2 and (h, limit) in worst:
            p = worst[(h, limit)]
        else:
            p = np.where(rng.random(len(p_lb)) < 0.5, p_lb, p_ub)
        u = follower(A, B, b, d1, d2, p)
        if u is None:
            continue
        items.append((h, int(rng.integers(N_STATES)), p, u))
    return items
```

genco_rss/benders.py

```
"""Two-phase Benders decomposition for a linear bilevel problem with an LP follower.

Python port of bilevel_benders.m (dissertation code, MATLAB):

    leader   : max_{p,u} c1'p + d1'u ,  p_lb <= p <= p_ub
    follower : u in argmax { d2'u : A p + B u <= b , u >= 0 }

Phase 1 master  : p, u and the complementarity pattern z (binary); exact
                  primal bounds; no dual variables; no big-M on the primal side.
Robust cut      : d2'u >= phi_R (one LP), plus the adversarial vertex p_adv.
Subproblem      : follower dual-feasibility LP -> combinatorial Benders cut.
Phase 2         : follower duals lifted into the master (big-M on duals only).
Every incumbent is the exact follower reaction at its p, so `obj` is always
attained and `bound` is a valid bound (certified gap = bound - obj).
"""
from __future__ import annotations

import time
from dataclasses import dataclass, field
```

```
import numpy as np
import scipy.sparse as sp
from scipy.optimize import Bounds, LinearConstraint, linprog, milp


@dataclass
class Options:
    tol_gap: float = 1e-3
    max_iter: int = 200
    time_limit: float = 300.0
    tol_u: float = 1e-6
    tol_slack: float = 1e-6
    robust_cut: bool = True
    local_search: bool = True
    local_search_gap: float = 1e-6
    local_search_passes: int = 2
    fallback_iter: int = 15
    dual_bound: float = 5e4
    verbose: bool = False


@dataclass
class Result:
    status: str
    obj: float
    bound: float
    p: np.ndarray
    u: np.ndarray
    iters: int
    cuts: int
    time: float
    phi_robust: float
    phase2: bool = False
    history: list = field(default_factory=list)

    @property
    def gap(self):
        return (self.bound - self.obj) / max(1.0, abs(self.bound))


def _lp(c, A=None, b=None, Aeq=None, beq=None, bounds=None):
    return linprog(c, A_ub=A, b_ub=b, A_eq=Aeq, b_eq=beq, bounds=bounds, method="highs")


def follower_response(A, B, b, c1, d1, d2, p):
    """Optimal (optimistic) follower reaction at p; returns (F, u, ok)."""
    nu = B.shape[1]
    rhs = b - A @ p
    r = _lp(-d2, B, rhs, bounds=[(0, None)] * nu)
    if r.status != 0:
        return -np.inf, None, False
    phi = -r.fun
    tol = 1e-9 * max(1.0, abs(phi))
    r2 = _lp(-d1, sp.vstack([B, -sp.csr_matrix(d2)]), np.r_[rhs, -(phi - tol)], bounds=[(0, None)] * nu)
    u = r2.x if r2.status == 0 else r.x
    return float(c1 @ p + d1 @ u), u, True


def robust_bound(A, B, b, d2, p_lb, p_ub, u_ub):
    """phi_R = max{d2'v : B v <= b - max_p A p for all p}; balancing units
    (d2 = 0, coupled only through one equality pair) may adapt. Returns
    (phi_R, p_adv) or (-inf, None) when no such static response exists."""
    A = sp.csr_matrix(A); B = sp.csr_matrix(B)
    m = A.shape[0]; nu = B.shape[1]
    F = np.where(d2 == 0)[0]
    X = np.setdiff1d(np.arange(nu), F)
    nzB = (B != 0).sum(1).A1; nzA = (A != 0).sum(1).A1
    own = (nzB == 1) & (nzA == 0)
    Bd = B.toarray(); Ad = A.toarray()
    wl = np.zeros(len(F)); wu = u_ub[F].astype(float).copy()
    for t, f in enumerate(F):
        for i in np.where(own & (Bd[:, f] != 0))[0]:
            a = Bd[i, f]
            if a > 0:
                wu[t] = min(wu[t], b[i] / a)
            else:
```

```
                wl[t] = max(wl[t], b[i] / a)
    coup = np.where((np.abs(Bd[:, F]).sum(1) > 0) & ~own)[0] if len(F) else np.array([], int)
    ok = True
    if len(coup):
        if len(coup) != 2 or np.any(np.hstack([Ad[coup[0]], Bd[coup[0]]]) + np.hstack([Ad[coup[1]],
Bd[coup[1]]])):
            ok = False
    if not ok or np.any(wl > wu):
        X = np.arange(nu); F = np.array([], int); coup = np.array([], int)
    absorb = np.zeros(m)
    for i in coup:
        bf = Bd[i, F]
        absorb[i] = np.sum(np.maximum(-bf * wl, -bf * wu))
    Amax = np.maximum(Ad, 0) @ p_ub + np.minimum(Ad, 0) @ p_lb
    ownF = own & (np.abs(Bd[:, F]).sum(1) > 0) if len(F) else np.zeros(m, bool)
    rows = np.where((np.abs(Bd[:, X]).sum(1) > 0) & ~ownF)[0]
    rhs = b[rows] - Amax[rows] + absorb[rows]
    r = _lp(-d2[X], Bd[np.ix_(rows, X)], rhs, bounds=list(zip(np.zeros(len(X)), u_ub[X])))
    if r.status != 0:
        return -np.inf, None
    mu = np.abs(r.ineqlin.marginals)
    g = Ad[rows].T @ mu
    return -r.fun, np.where(g > 0, p_ub, p_lb)


def _dual_feasible(BR, d2, Ju, JlR, nR):
    free = np.ones(nR, bool); free[JlR] = False
    idx = np.where(free)[0]
    if len(idx) == 0:
        return bool(np.all(d2 <= 1e-12) and np.all(np.abs(d2[Ju]) <= 1e-12))
    Bt = BR[idx].T.tocsr()
    r = _lp(np.ones(len(idx)), -Bt, -d2, Bt[Ju] if len(Ju) else None, d2[Ju] if len(Ju) else None,
            bounds=[(0, None)] * len(idx))
    return r.status == 0


def _farkas_subset(BR, d2, Ju, JlR, nR):
    nu = len(d2); nJu, nJl = len(Ju), len(JlR)
    Abase = sp.hstack([-BR.T, sp.csr_matrix((nu, nJl + nJu))])
    Al = sp.hstack([sp.csr_matrix((np.ones(nJl), (np.arange(nJl), JlR)), shape=(nJl, nR)),
                    -sp.identity(nJl), sp.csr_matrix((nJl, nJu))])
    Au = sp.hstack([BR[:, Ju].T, sp.csr_matrix((nJu, nJl)), -sp.identity(nJu)])
    r = _lp(np.r_[np.zeros(nR), np.ones(nJl + nJu)], sp.vstack([Abase, Al, Au]).tocsr(),
            np.r_[-d2, np.zeros(nJl), d2[Ju]], bounds=[(0, None)] * (nR + nJl + nJu))
    if r.status == 0 and r.fun > 1e-7:
        y = np.abs(r.ineqlin.marginals)
        Cl = JlR[y[nu:nu + nJl] > 1e-9]; Cu = Ju[y[nu + nJl:] > 1e-9]
        if (len(Cu) or len(Cl)) and not _dual_feasible(BR, d2, Cu, Cl, nR):
            return Cu, Cl
    return Ju, JlR


def local_search(best, A, B, b, c1, d1, d2, p_lb, p_ub, passes=2):
    """1-flip search over the leader box vertices to improve the incumbent."""
    F, p, u = best
    cand = np.where(p_ub - p_lb > 1e-12)[0]
    for _ in range(passes):
        improved = False
        for j in cand:
            q = p.copy()
            q[j] = p_ub[j] if abs(p[j] - p_lb[j]) <= abs(p[j] - p_ub[j]) else p_lb[j]
            Fq, uq, ok = follower_response(A, B, b, c1, d1, d2, q)
            if ok and Fq > F + 1e-9 * max(1, abs(Fq)):
                F, p, u = Fq, q, uq
                improved = True
        if not improved:
            break
    return (F, p, u)


def solve(A, B, b, c1, d1, d2, p_lb, p_ub, u_ub, opts: Options | None = None) -> Result:
    opts = opts or Options()
    t0 = time.time()
    A = sp.csr_matrix(A); B = sp.csr_matrix(B)
    b = np.asarray(b, float); m, n_p = A.shape; nu = B.shape[1]
    R = np.where(np.abs(B).sum(1).A1 > 0)[0]; nR = len(R); BR = B[R].tocsr()
```

```
    S = b - (A.maximum(0) @ p_lb + A.minimum(0) @ p_ub + B.minimum(0) @ u_ub)
    S = np.maximum(S, 0)[R]
    ip = np.arange(n_p); iu = n_p + np.arange(nu)
    izu = n_p + nu + np.arange(nu); izl = n_p + 2 * nu + np.arange(nR)
    nv = n_p + 2 * nu + nR
    f = np.zeros(nv); f[ip] = -c1; f[iu] = -d1
    Z = lambda r, c: sp.csr_matrix((r, c))
    rows = [sp.hstack([A, B, Z(m, nu + nR)]),
            sp.hstack([Z(nu, n_p), sp.identity(nu), -sp.diags(u_ub), Z(nu, nR)]),
            sp.hstack([-A[R], -B[R], Z(nR, nu), sp.diags(S)])]
    rhs = [b, np.zeros(nu), S - b[R]]
    K = sp.hstack([A[R], B[R]]).toarray()
    key = {}
    for a in range(nR):
        key.setdefault(K[a].tobytes(), a)
    I, J = [], []
    for a in range(nR):
        j = key.get((-K[a] + 0.0).tobytes())
        if j is not None and j > a and b[R[a]] + b[R[j]] > 1e-9:
            I.append(a); J.append(j)
    if I:
        rows.append(sp.csr_matrix((np.ones(2 * len(I)), (np.r_[np.arange(len(I)), np.arange(len(I))],
                                                         np.r_[izl[I], izl[J]])), shape=(len(I), nv)))
        rhs.append(np.ones(len(I)))
    phi_R, p_adv = (-np.inf, None)
    if opts.robust_cut:
        phi_R, p_adv = robust_bound(A, B, b, d2, p_lb, p_ub, u_ub)
        if np.isfinite(phi_R):
            r5 = np.zeros(nv); r5[iu] = -d2
            rows.append(sp.csr_matrix(r5)); rhs.append(np.array([-(phi_R - 1e-9 * max(1, abs(phi_R)))]))
    Am = sp.vstack(rows).tocsr(); bm = np.concatenate(rhs)
    lb = np.r_[p_lb, np.zeros(2 * nu + nR)]; ub = np.r_[p_ub, u_ub, np.ones(nu + nR)]
    integ = np.r_[np.zeros(n_p + nu), np.ones(nu + nR)]
    cuts_A, cuts_b = [], []
    best = (-np.inf, None, None)
    if p_adv is not None:
        F_, u_, ok = follower_response(A, B, b, c1, d1, d2, p_adv)
        if ok:
            best = (F_, p_adv, u_)
    UB = np.inf; status = "maxiter"; lifted = False; hist = []
    TolU = np.maximum(opts.tol_u, 1e-5 * u_ub); TolS = np.maximum(opts.tol_slack, 1e-5 * S)
    it = 0
    for it in range(1, opts.max_iter + 1):
        if time.time() - t0 > opts.time_limit:
            status = "timelimit"; break
        if not lifted and (it > opts.fallback_iter or (opts.robust_cut and not np.isfinite(phi_R))):
            nv0 = Am.shape[1]; il = nv0 + np.arange(nR)
            Am = sp.hstack([Am, Z(Am.shape[0], nR)]).tocsr()
            cuts_A = [sp.hstack([c, Z(1, nR)]).tocsr() for c in cuts_A]
            Ld = opts.dual_bound
            D1 = sp.hstack([Z(nu, nv0), -BR.T])
            D2 = sp.lil_matrix((nu, nv0 + nR)); D2[:, il] = BR.T; D2[:, izu] = Ld * sp.identity(nu)
            D3 = sp.lil_matrix((nR, nv0 + nR)); D3[:, il] = sp.identity(nR); D3[:, izl] = -Ld *
sp.identity(nR)
            Am = sp.vstack([Am, D1, D2.tocsr(), D3.tocsr()]).tocsr()
            bm = np.r_[bm, -d2, d2 + Ld, np.zeros(nR)]
            f = np.r_[f, np.zeros(nR)]; lb = np.r_[lb, np.zeros(nR)]; ub = np.r_[ub, np.full(nR, np.inf)]
            integ = np.r_[integ, np.zeros(nR)]; lifted = True
        Aall = sp.vstack([Am] + cuts_A).tocsr() if cuts_A else Am
        r = milp(f, constraints=LinearConstraint(Aall, -np.inf, np.r_[bm, cuts_b]), integrality=integ,
                 bounds=Bounds(lb, ub),
                 options={"mip_rel_gap": 1e-8, "time_limit": max(1.0, opts.time_limit - (time.time() - t0))})
        if r.x is None:
            status = "infeasible" if it == 1 else "master_failed"; break
        UB = min(UB, -r.fun)
        v = r.x; p_hat = v[ip]; u_hat = np.maximum(v[iu], 0)
        F_, u_, ok = follower_response(A, B, b, c1, d1, d2, p_hat)
        if ok and F_ > best[0]:
            best = (F_, p_hat, u_)
        if it == 1 and opts.local_search and best[1] is not None and (UB - best[0]) / max(1, abs(UB)) >
opts.local_search_gap:
            best = local_search(best, A, B, b, c1, d1, d2, p_lb, p_ub, opts.local_search_passes)
        gap = (UB - best[0]) / max(1, abs(UB))
        hist.append((it, UB, best[0]))
        if opts.verbose:
            print(f"  it {it:3d}  bound {UB:.6f}  incumbent {best[0]:.6f}  gap {gap:.2e}")
```

```
        if lifted or gap <= opts.tol_gap:
            status = "optimal" if gap <= 1e-6 else "gap"; break
        s = b - A @ p_hat - B @ u_hat
        Ju = np.where(u_hat > TolU)[0]; JlR = np.where(s[R] > TolS)[0]
        if _dual_feasible(BR, d2, Ju, JlR, nR):
            Fm = c1 @ p_hat + d1 @ u_hat
            if Fm >= best[0]:
                best = (Fm, p_hat, u_hat)
            status = "optimal"; break
        Cu, Cl = _farkas_subset(BR, d2, Ju, JlR, nR)
        zu = np.round(v[izu]); zl = np.round(v[izl])
        if not (np.all(zu[Cu] == 1) and np.all(zl[Cl] == 0)):
            Ju = np.where(zu == 1)[0]; JlR = np.where(zl == 0)[0]
            Cu, Cl = _farkas_subset(BR, d2, Ju, JlR, nR)
        row = sp.lil_matrix((1, Am.shape[1])); row[0, izu[Cu]] = 1; row[0, izl[Cl]] = -1
        cuts_A.append(row.tocsr()); cuts_b.append(len(Cu) - 1)
    return Result(status, best[0], UB, best[1], best[2], it, len(cuts_A), time.time() - t0, phi_R, lifted,
hist)
```

genco_rss/bess.py

```
"""Second-life EV battery fleet at the 20 PV buses (extension of the RSS).

Each PV bus hosts a retired-EV battery with nominal power 35 % of the bus PV
capacity and a 4 h energy rating, derated by its state of health (SoH). For
the one-hour RSS the usable power is P_eff = SoH * P_nom in both directions
(an SoC window [20 %, 85 %] starting at 50 % covers one hour at P_eff).
The batteries enter the bilevel model as follower (dispatch) variables.
"""
import numpy as np

SOH = np.array([0.84, 0.82, 0.79, 0.81, 0.76, 0.78, 0.75, 0.80, 0.74, 0.72] * 2)
POWER_RATIO = 0.35


def bess_power(grid, power_ratio=POWER_RATIO, soh=SOH):
    caps = np.repeat(grid.profile[0, [2, 4, 6, 8, 10]], 4) * 0.25 / grid.base
    caps[15] *= 2.0
    return soh[:len(caps)] * power_ratio * caps
```

genco_rss/conformal.py

```
"""Split-conformal security screening driven by GENCO's physics-residual trajectory.

Security quantities k (per scenario, worst over the network):
    load  : max branch apparent-power loading           (limit 1.0)
    vlow  : max_i (Vmin_i - Vm_i) over PQ buses         (limit V_TOL)
    vhigh : max_i (Vm_i - Vmax_i) over PQ buses         (limit V_TOL)

Three screening rules, all with the same finite-sample guarantee:

  plain     q_hat = GENCO prediction,            score s = q - q_hat
  normalised q_hat = GENCO prediction,           score s = (q - q_hat) / (sigma + beta r_K)
  ptr       q_hat = GENCO prediction + g_k(z),   score s = q - q_hat
            "physics-trajectory recalibration": z = GENCO's own outputs and the
            per-layer AC power-mismatch trajectory r_1..r_K, rho = r_K / r_1 that
            GENCO computes inside its corrective layers (no extra cost); g_k is a
            gradient-boosted regressor of GENCO's error fitted on the TUNE split.

With n calibration points and level a_k = alpha / 3, the threshold c_k is the
ceil((n+1)(1-a_k))-th smallest calibration score. A scenario is flagged if
    upper_k = q_hat_k + c_k * scale_k > limit_k   for some k.

Guarantee (calibration and test scenarios exchangeable; g_k, sigma_k, beta_k fixed
before calibration, i.e. fitted on an independent split):
    P(insecure and not flagged) <= sum_k a_k = alpha.
Proof: insecure => q_k > limit_k for some k; not flagged => upper_k <= limit_k,
hence s_k > c_k, an event of probability <= a_k by split-conformal exchangeability.
"""
```

```python
from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np
import pandas as pd

V_TOL = 0.01      # p.u. tolerance on bus-voltage limits (set-point artefact of <= 0.0055 p.u. in the data)
QUANTS = {"load": 1.0, "vlow": V_TOL, "vhigh": V_TOL}
METHODS = ("plain", "normalised", "ptr")


def true_insecure(df: pd.DataFrame) -> np.ndarray:
    return np.any([df[f"{k}_true"].values > lim for k, lim in QUANTS.items()], axis=0)


def trajectory_features(df: pd.DataFrame) -> list[str]:
    """GENCO-only features: predictions + per-layer residual trajectory (no hour/topology)."""
    rk = sorted([c for c in df.columns if c[0] == "r" and c[1:].isdigit()], key=lambda c: int(c[1:]))
    return [f"{k}_pred" for k in QUANTS] + ["qex_pred", "slack_pred"] + rk + ["rho"]


def _thr(scores, a):
    n = len(scores)
    r = int(np.ceil((n + 1) * (1 - a)))
    if r > n:
        return np.inf
    return np.sort(scores)[r - 1]


@dataclass
class Screen:
    alpha: float = 0.05
    method: str = "ptr"
    features: list | None = None
    sigma: dict = field(default_factory=dict)
    beta: dict = field(default_factory=dict)
    c: dict = field(default_factory=dict)
    model: dict = field(default_factory=dict)

    # ------------------------------------------------------------------ pieces
    def _scale(self, df, k):
        return self.sigma[k] + self.beta[k] * df["r_last"].values

    def predict(self, df, k):
        q = df[f"{k}_pred"].values.astype(float)
        if self.method == "ptr":
            q = q + self.model[k].predict(df[self.features].values)
        return q

    # ------------------------------------------------------------------ fit
    def fit(self, tune: pd.DataFrame, calib: pd.DataFrame, seed: int = 0):
        assert self.method in METHODS, self.method
        a = self.alpha / len(QUANTS)
        if self.method == "ptr":
            from sklearn.ensemble import GradientBoostingRegressor
            self.features = self.features or trajectory_features(tune)
        for k in QUANTS:
            err = (tune[f"{k}_true"] - tune[f"{k}_pred"]).values
            self.sigma[k], self.beta[k] = 1.0, 0.0
            if self.method == "ptr":
                self.model[k] = GradientBoostingRegressor(
                    n_estimators=300, max_depth=3, learning_rate=0.05, subsample=0.8,
                    random_state=seed).fit(tune[self.features].values, err)
            elif self.method == "normalised":
                s0 = max(np.quantile(np.abs(err), 0.5), 1e-4)
                rl = tune["r_last"].values
                best = (np.inf, s0, 0.0)
                for sig in s0 * np.array([0.25, 0.5, 1.0, 2.0]):
                    for bet in np.array([0.0, 0.25, 0.5, 1.0, 2.0, 4.0]) * s0 / max(np.median(rl), 1e-9):
                        sc = err / (sig + bet * rl)
                        width = np.mean(_thr(sc, a) * (sig + bet * rl))
                        if width < best[0]:
                            best = (width, sig, bet)
                self.sigma[k], self.beta[k] = best[1], best[2]
            sc = (calib[f"{k}_true"].values - self.predict(calib, k)) / self._scale(calib, k)
```

```
            self.c[k] = _thr(sc, a)
        return self

    # ------------------------------------------------------------------ use
    def upper(self, df, k):
        return self.predict(df, k) + self.c[k] * self._scale(df, k)

    def flag(self, df: pd.DataFrame) -> np.ndarray:
        return np.any([self.upper(df, k) > lim for k, lim in QUANTS.items()], axis=0)

    def mae(self, df: pd.DataFrame, k: str = "load") -> float:
        return float(np.abs(df[f"{k}_true"].values - self.predict(df, k)).mean())

    def report(self, df: pd.DataFrame) -> dict:
        f = self.flag(df)
        ins = true_insecure(df)
        return {"n": len(df), "insecure": int(ins.sum()), "flagged": int(f.sum()),
                "flag_rate": float(f.mean()), "missed": int((ins & ~f).sum()),
                "miss_rate_of_insecure": float((ins & ~f).sum() / max(ins.sum(), 1)),
                "miss_rate": float((ins & ~f).mean()),
                "exact_ac_saved": float(1 - f.mean()),
                "secure_cleared_frac": float((~f & ~ins).sum() / max((~ins).sum(), 1)),
                "load_MAE_after": self.mae(df)}
```

genco_rss/datagen.py

```
"""Exact AC power-flow data for GENCO with gridfm-datakit.

gridfm-datakit (PF mode) normally sets generator set-points with an AC-OPF in
Julia/PowerModels. Here that single step is replaced by an operating-point
source tied to the robust-secure-set (RSS) problem; everything else is
datakit (lightsim2grid Newton-Raphson AC power flow, DC-PF baseline, the
parquet layout that gridfm-graphkit reads, runtime logging).

Two sources:
  * RandomRSS  - training data: hours 1..24, topology (base / 69-70 / 100-103),
                 PV/load vertices or interior points of the hourly box, and
                 DC-feasible vertex or interior dispatches inside ramp limits.
  * ScenarioList - a given list of (hour, state, p, u): used to verify
                 Benders / adversarial operating points with exact AC.

Every consumed operating point is recorded in meta.npz next to the parquet
files, keyed by datakit's scenario index.

Usage:
    python -m genco_rss.datagen --scenarios 20000 --out data/train --seed 0
"""
from __future__ import annotations

import argparse
import os

import numpy as np
from scipy.optimize import linprog

from .grid import CASE_FILE, Grid, contingency_mask
from .rss import dc_opf
from .vsetpoints import gen_vm, setpoints


class RandomRSS:
    def __init__(self, grid: Grid, seed: int = 0):
        self.grid = grid
        self.rng = np.random.default_rng(seed)
        self.cache = {}

    def _limits(self, h):
        if h not in self.cache:
            p_lb, p_ub, p_mid = self.grid.scenario_box(h)
            u_lb, u_ub = self.grid.gen_limits(0.3, dc_opf(self.grid, p_mid))
            self.cache[h] = (p_lb, p_ub, u_lb, u_ub)
        return self.cache[h]
```

```
    def __call__(self):
        g, rng = self.grid, self.rng
        for _ in range(100):
            h = int(rng.integers(1, 25))
            state = int(rng.choice([0, 0, 1, 2]))
            p_lb, p_ub, u_lb, u_ub = self._limits(h)
            if rng.random() < 0.5:
                p = np.where(rng.random(len(p_lb)) < 0.5, p_lb, p_ub)
            else:
                p = p_lb + rng.random(len(p_lb)) * (p_ub - p_lb)
            A, B, b = g.rss_matrices(u_lb, u_ub)
            c = rng.normal(size=B.shape[1])
            if rng.random() < 0.3:          # RSS follower objectives (max / min sum of outputs)
                d = np.ones(B.shape[1]); d[g.slack_gen] = 0.0
                c = rng.choice([-1.0, 1.0]) * d + 0.05 * c
            r = linprog(c, A_ub=B, b_ub=b - A @ p, bounds=[(0, None)] * B.shape[1], method="highs")
            if r.status != 0:
                continue
            u = r.x
            if rng.random() < 0.4:
                r2 = linprog(-c, A_ub=B, b_ub=b - A @ p, bounds=[(0, None)] * B.shape[1], method="highs")
                if r2.status == 0:
                    lam = rng.random()
                    u = lam * r.x + (1 - lam) * r2.x
            return h, state, p, u
        raise RuntimeError("could not sample a DC-feasible operating point")


class ScenarioList:
    def __init__(self, items):
        self.items = list(items)
        self.k = 0

    def __call__(self):
        it = self.items[self.k % len(self.items)]
        self.k += 1
        return it


def apply_operating_point(net, grid: Grid, state, p, u, vg=None):
    """Write (state, p, u) into a datakit Network; returns Pg (MW) per gen.
    vg: generator voltage set-points (per gen row) of the hour (AC-OPF)."""
    base = grid.base
    bus = grid.case["bus"]
    pd = bus[:, 2].copy(); qd = bus[:, 3].copy()
    ratio = np.divide(qd, pd, out=np.zeros_like(qd), where=pd > 0)
    pd_new = pd.copy()
    pd_new[grid.load_bus] = p[grid.npv:] * base
    qd_new = qd.copy()
    qd_new[grid.load_bus] = pd_new[grid.load_bus] * ratio[grid.load_bus]   # constant power factor
    pd_new[grid.pv_bus] -= p[:grid.npv] * base                              # PV at unity power factor
    net.Pd = pd_new
    net.Qd = qd_new
    if state > 0:
        from gridfm_datakit.utils.idx_brch import BR_B, BR_R, BR_X, F_BUS, T_BUS
        br = net.branches
        # the network keeps its internal 0-based bus indices; map the pglib numbers
        orig = grid.case["branch"]
        hit = contingency_mask(orig, state)
        ia, ib = grid.fr[hit], grid.to[hit]
        sel = np.zeros(len(br), bool)
        for a, b_ in zip(ia, ib):
            sel |= ((br[:, F_BUS] == a) & (br[:, T_BUS] == b_)) | ((br[:, F_BUS] == b_) & (br[:, T_BUS] ==
a))
        br[sel, BR_X] *= 2.0
        br[sel, BR_R] *= 2.0
        br[sel, BR_B] *= 0.5
    ng = len(grid.disp)
    if len(u) > ng:                                  # batteries at the PV buses (discharge - charge)
        nbat = (len(u) - ng) // 2
        inj = u[ng:ng + nbat] - u[ng + nbat:]
        pd_new[grid.pv_bus[:nbat]] -= inj * base
        net.Pd = pd_new
        u = u[:ng]
    if vg is not None:
```

```
        from gridfm_datakit.utils.idx_gen import VG
        net.gens[:, VG] = vg
    pg = np.zeros(len(grid.case["gen"]))
    pg[grid.disp] = u * base
    return pg


def install_patches(grid: Grid, source):
    """Replace datakit's Julia steps by the operating-point source (pf_solver: lightsim2grid)."""
    import gridfm_datakit.generate as gen_mod
    import gridfm_datakit.network as net_mod
    import gridfm_datakit.perturbations.load_perturbation as lp
    import gridfm_datakit.process.process_network as pn
    from gridfm_datakit.utils.idx_gen import GEN_BUS, VG

    gen_mod.init_julia = lambda *a, **k: None
    net_mod.correct_network = lambda path, *a, **k: path        # PGLib file is already clean

    class _Done:
        def get(self, timeout=None):
            return 1.0

    class _Pool:
        def close(self):
            pass

        def join(self):
            pass

    lp.LoadScenarioGeneratorBase.find_largest_scaling_factor = lambda self, *a, **k: (_Pool(), _Done())
    log = []
    vsp = setpoints(grid)

    def run_opf_rss(net, jl):
        h, state, p, u = source()
        log.append((h, state, p, u))
        pg = apply_operating_point(net, grid, state, p, u, gen_vm(grid, vsp[int(h)]))
        ids = net.idx_gens_in_service
        gen_pq = np.zeros((len(ids), 2))
        gen_pq[:, 0] = pg[ids] / net.baseMVA
        vm = np.ones(net.buses.shape[0])
        for gi in ids:
            vm[int(net.gens[gi, GEN_BUS])] = net.gens[gi, VG]
        flows = np.zeros((len(net.idx_branches_in_service), 4))
        return {"termination_status": "LOCALLY_SOLVED",
                "solution": {"arrays": (flows, gen_pq, np.c_[vm, np.zeros_like(vm)])}}

    pn.run_opf = run_opf_rss
    return log


def config(scenarios: int, data_dir: str, seed: int = 0) -> dict:
    return {
        "network": {"name": "case118_ieee", "source": "file", "network_dir": os.path.dirname(CASE_FILE)},  #
data/case118_ieee.m = PGLib v23.07
        "load": {"generator": "agg_load_profile", "agg_profile": "default", "scenarios": scenarios,
                 "sigma": 0.05, "change_reactive_power": True, "global_range": 0.2,
                 "max_scaling_factor": 1.2, "step_size": 0.1, "start_scaling_factor": 0.8},
        "topology_perturbation": {"type": "none"},
        "generation_perturbation": {"type": "none"},
        "admittance_perturbation": {"type": "none"},
        "settings": {"num_processes": 1, "data_dir": data_dir, "large_chunk_size": 500,
                     "overwrite": True, "mode": "pf", "include_dc_res": True,
                     "enable_solver_logs": False, "pf_fast": True, "dcpf_fast": True,
                     "max_iter": 200, "seed": seed, "pf_solver": "lightsim2grid"},
    }


def _stack(arrs):
    """Rows of possibly different length -> NaN-padded float matrix."""
    n = max(len(a) for a in arrs)
    out = np.full((len(arrs), n), np.nan)
    for i, a in enumerate(arrs):
        out[i, :len(a)] = a
    return out
```

```
def generate(grid: Grid, source, n: int, out: str, seed: int = 0):
    """Run gridfm-datakit for n operating points from `source` into out/case118_ieee/raw."""
    log = install_patches(grid, source)
    from gridfm_datakit.generate import generate_power_flow_data
    os.makedirs(out, exist_ok=True)
    paths = generate_power_flow_data(config(n, os.path.abspath(out), seed))
    raw = os.path.dirname(paths["bus_data"])
    np.savez_compressed(os.path.join(raw, "meta.npz"),
                        hour=np.array([x[0] for x in log]), state=np.array([x[1] for x in log]),
                        p=_stack([x[2] for x in log]), u=_stack([x[3] for x in log]))
    return raw


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--scenarios", type=int, default=2000)
    ap.add_argument("--out", default="data/train")
    ap.add_argument("--seed", type=int, default=0)
    a = ap.parse_args()
    grid = Grid.load()
    print(generate(grid, RandomRSS(grid, a.seed), a.scenarios, a.out, a.seed))


if __name__ == "__main__":
    main()
```

genco_rss/genco.py

```
"""GENCO in the loop: inference, physics-residual probe and gradients.

Everything goes through gridfm-graphkit's own pipeline (config loader,
HeteroGridDatasetDisk, task transforms/masks, normalizer, GNS_heterogeneous,
ComputeBranchFlow / ComputeNodeResiduals), so the model sees exactly the
inputs it was trained on.

* `Genco.evaluate(root)`  per-scenario GENCO predictions of the security
  quantities (max branch loading in MVA, PQ-bus voltage range, reactive
  excess, slack output), the same quantities from the exact AC solution
  stored by gridfm-datakit, and GENCO's internal power-mismatch trajectory
  r^(1..K) captured with a forward hook on the model's residual layer.
* `Genco.adversarial(...)`  projected sign-gradient ascent of a smooth
  maximum of GENCO-predicted branch loading over the PV/load box, with the
  three topologies evaluated jointly (autograd through GENCO).
"""
from __future__ import annotations

import os
import time
from types import SimpleNamespace

import numpy as np
import pandas as pd
import torch
from torch_geometric.loader import DataLoader

from gridfm_graphkit.cli import _load_config, _normalize_loaded_state_dict_keys
from gridfm_graphkit.datasets.globals import (MAX_PG, MAX_QG_H, MAX_VM_H, MIN_QG_H, MIN_VM_H, PD_H, PG_OUT,
                                              QD_H, QG_OUT, RATE_A, VM_OUT)
from gridfm_graphkit.datasets.masking import RemovePFMask
from gridfm_graphkit.datasets.powergrid_hetero_dataset import HeteroGridDatasetDisk
from gridfm_graphkit.io.param_handler import get_task, get_task_transforms, load_normalizer
from gridfm_graphkit.models.utils import ComputeBranchFlow, ComputeNodeInjection, ComputeNodeResiduals
from gridfm_graphkit.tasks.pf_task import _build_bus_target, _clamp_known_to_ground_truth

try:
    from torch_scatter import scatter_max, scatter_mean, scatter_min
except ImportError:  # pragma: no cover
    raise

EDGE = ("bus", "connects", "bus")
```

```python
class Genco:
    def __init__(self, config: str, model_path: str, stats_path: str, device: str = "cpu"):
        self.cfg = _load_config(SimpleNamespace(config=config, config_conversion="no"))
        self.norm = load_normalizer(self.cfg)
        stats = torch.load(stats_path, map_location="cpu", weights_only=True)
        net = self.cfg.data.networks[0]
        self.norm.fit_from_dict(stats[net] if net in stats else stats)
        self.task = get_task(self.cfg, [self.norm])
        sd = torch.load(model_path, map_location="cpu")
        self.task.load_state_dict(_normalize_loaded_state_dict_keys(sd))
        self.model = self.task.model.to(device).eval()
        self.device = device
        self._res = []
        self.model.node_residuals_layer.register_forward_hook(self._hook)
        self.n_params = sum(p.numel() for p in self.model.parameters())

    # ------------------------------------------------------------------ probe
    def _hook(self, module, inp, out):
        rp, rq = out
        self._res.append(torch.sqrt(rp.detach() ** 2 + rq.detach() ** 2))

    def dataset(self, root: str):
        """root = <data_path>/<network> containing raw/*.parquet from gridfm-datakit."""
        return HeteroGridDatasetDisk(root=root, data_normalizer=self.norm,
                                     transform=get_task_transforms(self.cfg))

    # ------------------------------------------------------------- evaluation
    @torch.no_grad()
    def evaluate(self, root: str, batch_size: int = 128) -> pd.DataFrame:
        ds = self.dataset(root)
        loader = DataLoader(ds, batch_size=batch_size, shuffle=False)
        rows, t_model = [], 0.0
        for batch in loader:
            batch = batch.to(self.device)
            self._res = []
            t0 = time.perf_counter()
            out = self.model(batch)
            t_model += time.perf_counter() - t0
            bb = batch.batch_dict["bus"]
            ng = int(bb.max()) + 1
            traj = torch.stack([scatter_mean(r, bb, dim=0, dim_size=ng) for r in self._res], 1)  # [G, K]
            rows.append(self._metrics(batch, out, traj))
        df = pd.concat(rows, ignore_index=True)
        df.attrs["model_seconds"] = t_model
        df.attrs["n"] = len(df)
        return df

    def _metrics(self, batch, out, traj):
        self.norm.inverse_transform(batch)
        self.norm.inverse_output(out, batch)
        RemovePFMask()(batch)
        nb = batch.x_dict["bus"].size(0)
        ei = batch.edge_index_dict[EDGE]
        ea = batch.edge_attr_dict[EDGE]
        target, g2b, _ = _build_bus_target(batch, nb)
        ev = _clamp_known_to_ground_truth(out["bus"], target, batch, g2b, nb)
        bb = batch.batch_dict["bus"]
        ng = int(bb.max()) + 1
        x = batch.x_dict["bus"]
        res = {}
        for tag, state in (("pred", ev), ("true", target)):
            P, Q = ComputeBranchFlow()(state, ei, ea)
            rate = ea[:, RATE_A]
            load = torch.where(rate > 0, torch.sqrt(P ** 2 + Q ** 2) / rate.clamp(min=1e-6),
torch.zeros_like(P))
            res[f"load_{tag}"] = scatter_max(load, bb[ei[0]], dim=0, dim_size=ng)[0]
            pq = batch.mask_dict["PQ"]
            vm = state[:, VM_OUT]
            big = torch.full_like(vm, 9.0)
            res[f"vmin_{tag}"] = scatter_min(torch.where(pq, vm, big), bb, dim=0, dim_size=ng)[0]
            res[f"vmax_{tag}"] = scatter_max(torch.where(pq, vm, -big), bb, dim=0, dim_size=ng)[0]
            res[f"vlow_{tag}"] = scatter_max(torch.where(pq, x[:, MIN_VM_H] - vm, -big), bb, dim=0,
dim_size=ng)[0]
            res[f"vhigh_{tag}"] = scatter_max(torch.where(pq, vm - x[:, MAX_VM_H], -big), bb, dim=0,
dim_size=ng)[0]
```

```
            gen_bus = batch.mask_dict["PV"] | batch.mask_dict["REF"]
            qg = state[:, QG_OUT]
            qex = torch.clamp(torch.maximum(qg - x[:, MAX_QG_H], x[:, MIN_QG_H] - qg), min=0.0)
            res[f"qex_{tag}"] = scatter_max(torch.where(gen_bus, qex, torch.zeros_like(qex)), bb, dim=0,
dim_size=ng)[0]
            ref = batch.mask_dict["REF"]
            res[f"slack_{tag}"] = scatter_max(torch.where(ref, state[:, PG_OUT], -1e9 * torch.ones_like(vm)),
bb, dim=0, dim_size=ng)[0]
        # GENCO's own physics check of its final prediction (MVA mismatch per bus)
        P, Q = ComputeBranchFlow()(ev, ei, ea)
        Pin, Qin = ComputeNodeInjection()(P, Q, ei, nb)
        rP, rQ = ComputeNodeResiduals()(Pin, Qin, ev, x)
        res["pbe_mean"] = scatter_mean(torch.sqrt(rP ** 2 + rQ ** 2), bb, dim=0, dim_size=ng)
        res["vm_err_max"] = scatter_max(torch.where(batch.mask_dict["PQ"], (ev[:, VM_OUT] - target[:,
VM_OUT]).abs(),
                                                    torch.zeros_like(x[:, 0])), bb, dim=0, dim_size=ng)[0]
        df = pd.DataFrame({k: v.cpu().numpy() for k, v in res.items()})
        df["scenario"] = batch["scenario_id"].cpu().numpy()
        r = traj.cpu().numpy()
        df["r_first"] = r[:, 0]
        df["r_last"] = r[:, -1]
        df["rho"] = r[:, -1] / np.maximum(r[:, 0], 1e-12)
        for k in range(r.shape[1]):
            df[f"r{k + 1}"] = r[:, k]
        return df

    # ------------------------------------------------------------ gradients
    def adversarial(self, ds, idx_states, grid, p0, p_lb, p_ub, steps=8, step_frac=0.25, tau=0.02):
        """Projected sign-gradient ascent on p of a smooth max of predicted
        branch loading over the graphs ds[idx] (same p and dispatch, one per
        topology). Returns (p_adv, history of the smooth objective)."""
        batch0 = next(iter(DataLoader([ds[i] for i in idx_states], batch_size=len(idx_states))))
        bb = batch0.batch_dict["bus"]
        loc = torch.arange(bb.numel()) - torch.cat([torch.zeros(1, dtype=torch.long),
                                                    torch.cumsum(torch.bincount(bb), 0)[:-1]])[bb]
        s = float(self.norm.baseMVA)
        base = grid.base
        bus = grid.case["bus"]
        ratio = np.divide(bus[:, 3], bus[:, 2], out=np.zeros(len(bus)), where=bus[:, 2] > 0)
        Mpd = np.zeros((grid.nb, len(p0)))                    # Pd_bus [MW] = Mpd @ p  (+ constant)
        Mpd[grid.pv_bus, np.arange(grid.npv)] = -base
        Mpd[grid.load_bus, grid.npv + np.arange(len(grid.load_bus))] = base
        Mqd = np.zeros_like(Mpd)
        Mqd[grid.load_bus, grid.npv + np.arange(len(grid.load_bus))] = base * ratio[grid.load_bus]
        Mpd_t = torch.tensor(Mpd, dtype=torch.float32)[loc] / s
        Mqd_t = torch.tensor(Mqd, dtype=torch.float32)[loc] / s
        pd0 = batch0.x_dict["bus"][:, PD_H].clone() - Mpd_t @ torch.tensor(p0, dtype=torch.float32)
        qd0 = batch0.x_dict["bus"][:, QD_H].clone() - Mqd_t @ torch.tensor(p0, dtype=torch.float32)
        rate = batch0[EDGE].static[:, 2] / s                  # RATE_A column of .static, normalised
        lo = torch.tensor(p_lb, dtype=torch.float32); hi = torch.tensor(p_ub, dtype=torch.float32)
        p = torch.tensor(p0, dtype=torch.float32)
        hist = []
        for _ in range(steps):
            p = p.detach().requires_grad_(True)
            b = batch0.clone()
            xb = b.x_dict["bus"].clone()
            xb[:, PD_H] = pd0 + Mpd_t @ p
            xb[:, QD_H] = qd0 + Mqd_t @ p
            b["bus"].x = xb
            out = self.model(b)
            nb = xb.size(0)
            target, g2b, _ = _build_bus_target(b, nb)
            ev = _clamp_known_to_ground_truth(out["bus"], target, b, g2b, nb)
            P, Q = ComputeBranchFlow()(ev, b.edge_index_dict[EDGE], b.edge_attr_dict[EDGE])
            load = torch.sqrt(P ** 2 + Q ** 2 + 1e-12) / rate.clamp(min=1e-6)
            load = torch.where(rate > 0, load, torch.zeros_like(load))
            J = tau * torch.logsumexp(load / tau, 0)
            g, = torch.autograd.grad(J, p)
            hist.append(float(J.detach()))
            with torch.no_grad():
                p = torch.clamp(p + step_frac * (hi - lo) * torch.sign(g), lo, hi)
        self._res = []
        return p.detach().numpy().astype(float), hist


def load_latest(run_dir: str, config: str, device="cpu") -> Genco:
```

```
    """Load the best checkpoint + normalizer stats of a gridfm-graphkit MLflow run."""
    model = stats = None
    for root, _, files in os.walk(run_dir):
        for f in files:
            if f == "best_model_state_dict.pt" or (f == "last.pt" and model is None):
                model = os.path.join(root, f)
            if f == "normalizer_stats.pt":
                stats = os.path.join(root, f)
    if model is None or stats is None:
        raise FileNotFoundError(f"no checkpoint/normalizer_stats under {run_dir}")
    return Genco(config, model, stats, device)
```

genco_rss/grid.py

```
"""IEEE-118 (PGLib v23.07) grid with the uncertainty model of the dissertation.

* DC model: PTDF for the base case and the two contingencies of
  fault_data_Nki.m (one circuit of the double corridors 69-70 and 100-103
  lost, i.e. reactance doubled - reduce_dc.m).
* Uncertainty (Set_data.m): 20 PV buses grouped in 5 sites x 4 buses with
  25 % of the site capacity / forecast; PV error 30 %*h/24 of capacity,
  load error 3 %*h/24, PV only between hours 7 and 18.
* RSS bilevel (RSS_MILP2.m): nature chooses p = [P_PV; P_load] in the hourly
  box, the dispatch u follows; A p + B u <= b stacks line limits for the 3
  topologies, the power balance, and generator bounds.
"""
from __future__ import annotations

import os
import re
from dataclasses import dataclass, field

import numpy as np
import scipy.sparse as sp

HERE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(HERE, "data")
CASE_FILE = os.path.join(DATA, "pglib_opf_case118_ieee.m")

PV_BUSES = [5, 11, 14, 16, 17, 20, 23, 29, 33, 35, 37, 41, 45, 48, 53, 67, 75, 83, 94, 98]
CONTINGENCIES = [(69, 70), (100, 103)]          # fault_data_Nki.m (used for training and the RSS)
UNSEEN = [(49, 66)]                             # held-out topology, only for the GENCO probe study
STATE_NAMES = ["base", "69-70", "100-103", "49-66 (unseen)"]


def read_matpower(path: str) -> dict:
    txt = open(path).read()
    out = {"baseMVA": float(re.search(r"mpc\.baseMVA\s*=\s*([\d.]+)", txt).group(1))}
    for key in ("bus", "gen", "branch", "gencost"):
        m = re.search(r"mpc\.%s\s*=\s*\[(.*?)\];" % key, txt, re.S)
        rows = []
        for line in m.group(1).splitlines():
            line = line.split("%")[0].strip().rstrip(";")
            if line:
                rows.append([float(v) for v in line.split()])
        out[key] = np.array(rows)
    return out


def ptdf(nb: int, ref: int, fr: np.ndarray, to: np.ndarray, x: np.ndarray) -> np.ndarray:
    """Branch-by-bus PTDF with a zero column at the reference bus (0-based)."""
    nl = len(fr)
    Cf = sp.csr_matrix((np.ones(nl), (np.arange(nl), fr)), shape=(nl, nb))
    Ct = sp.csr_matrix((np.ones(nl), (np.arange(nl), to)), shape=(nl, nb))
    C = (Cf - Ct).tocsr()
    Bf = sp.diags(1.0 / x) @ C
    Bbus = (C.T @ Bf).toarray()
    keep = np.setdiff1d(np.arange(nb), [ref])
    H = np.zeros((nl, nb))
    H[:, keep] = (Bf.toarray()[:, keep]) @ np.linalg.inv(Bbus[np.ix_(keep, keep)])
    return H
```

```
def contingency_mask(branch: np.ndarray, state: int) -> np.ndarray:
    """Boolean mask of the branches whose reactance is doubled in `state`."""
    if state == 0:
        return np.zeros(len(branch), bool)
    a, b = (CONTINGENCIES + UNSEEN)[state - 1]
    return ((branch[:, 0] == a) & (branch[:, 1] == b)) | ((branch[:, 0] == b) & (branch[:, 1] == a))


@dataclass
class Grid:
    case: dict
    nb: int = 0
    ref: int = 0
    H: list = field(default_factory=list)

    @classmethod
    def load(cls, path: str | None = None) -> "Grid":
        c = read_matpower(path or CASE_FILE)
        g = cls(case=c)
        bus = c["bus"]
        g.base = c["baseMVA"]
        g.bus_ids = bus[:, 0].astype(int)
        g.idx = {b: i for i, b in enumerate(g.bus_ids)}
        g.nb = len(bus)
        g.ref = int(np.where(bus[:, 1] == 3)[0][0])
        gen = c["gen"]
        g.gen_bus = np.array([g.idx[int(b)] for b in gen[:, 0]])
        g.disp = np.where((gen[:, 8] > 0) & (gen[:, 7] > 0))[0]          # dispatchable units
        g.slack_gen = [k for k, gi in enumerate(g.disp) if g.gen_bus[gi] == g.ref][0]
        br = c["branch"]
        g.fr = np.array([g.idx[int(b)] for b in br[:, 0]])
        g.to = np.array([g.idx[int(b)] for b in br[:, 1]])
        g.x = br[:, 3].copy()
        g.rate = br[:, 5] / g.base
        g.H = []
        for s in range(1 + len(CONTINGENCIES)):
            xs = g.x.copy()
            xs[contingency_mask(br, s)] *= 2.0
            g.H.append(ptdf(g.nb, g.ref, g.fr, g.to, xs))
        g.pv_bus = np.array([g.idx[b] for b in PV_BUSES])
        g.load_bus = np.where(bus[:, 2] > 0)[0]
        g.profile = np.loadtxt(os.path.join(DATA, "load_pv_profile.csv"), delimiter=",", skiprows=1)
        return g

    # ------------------------------------------------------------- hourly box
    def scenario_box(self, hour: int):
        """(p_lb, p_ub, p_mid) for p = [PV (20); loads (n_load)] in p.u."""
        row = self.profile[hour - 1]
        load_factor = row[1] / self.profile[:, 1].max()
        pd = self.case["bus"][self.load_bus, 2] * load_factor / self.base
        eload = 0.03 * hour / 24.0
        caps = np.repeat(row[[2, 4, 6, 8, 10]], 4) * 0.25 / self.base
        caps[15] *= 2.0                                  # the single 50 % entry of Set_data.m
        pred = np.repeat(row[[3, 5, 7, 9, 11]], 4) * 0.25 / self.base
        epv = 3 * 0.10 * hour / 24.0
        pv_lo = np.clip(pred - epv * caps, 0, caps)
        pv_hi = np.clip(pred + epv * caps, 0, caps)
        if hour < 7 or hour > 18:
            pv_lo[:] = 0.0
            pv_hi[:] = 0.0
        p_lb = np.r_[pv_lo, pd * (1 - eload)]
        p_ub = np.r_[pv_hi, pd * (1 + eload)]
        return p_lb, p_ub, 0.5 * (p_lb + p_ub)

    @property
    def npv(self):
        return len(self.pv_bus)

    def injections(self, p):
        """Net bus withdrawal (load - PV) in p.u. for p = [PV; loads]."""
        pd = np.zeros(self.nb)
        pd[self.load_bus] += p[self.npv:]
        pd[self.pv_bus] -= p[:self.npv]
        return pd
```

```
    # ------------------------------------------------------------ RSS matrices
    def rss_matrices(self, u_lb, u_ub, rate_scale=None):
        """A p + B u <= b with the same row structure as RSS_MILP2.m.
        rate_scale: optional (n_state, n_line) derating factors (AC cuts)."""
        npv, nload = self.npv, len(self.load_bus)
        gb = self.gen_bus[self.disp]
        Ap, Bp, bp = [], [], []
        for s, Hs in enumerate(self.H):
            Ap.append(np.hstack([Hs[:, self.pv_bus], -Hs[:, self.load_bus]]))
            Bp.append(Hs[:, gb])
            r = self.rate if rate_scale is None else self.rate * rate_scale[s]
            bp.append(r)
        A_line, B_line, b_line = np.vstack(Ap), np.vstack(Bp), np.concatenate(bp)
        ng = len(gb)
        A_bal = np.r_[np.ones(npv), -np.ones(nload)][None, :]
        A = np.vstack([A_line, -A_line, A_bal, -A_bal, np.zeros((2 * ng, npv + nload))])
        B = np.vstack([B_line, -B_line, np.ones((1, ng)), -np.ones((1, ng)), -np.eye(ng), np.eye(ng)])
        b = np.r_[b_line, b_line, 0.0, 0.0, -u_lb, u_ub]
        return sp.csr_matrix(A), sp.csr_matrix(B), b

    def gen_limits(self, ramp_frac=0.3, p0=None):
        gen = self.case["gen"][self.disp]
        pmax, pmin = gen[:, 8] / self.base, gen[:, 9] / self.base
        if p0 is None:
            return pmin, pmax
        r = ramp_frac * pmax
        lb, ub = np.maximum(pmin, p0 - r), np.minimum(pmax, p0 + r)
        lb[self.slack_gen], ub[self.slack_gen] = pmin[self.slack_gen], pmax[self.slack_gen]
        return lb, ub

    def dc_flows(self, p, u, state):
        """DC branch flows (p.u.) of operating point (p, u) in topology `state`."""
        inj = -self.injections(p)
        np.add.at(inj, self.gen_bus[self.disp], u)
        return self.H[state] @ inj
```

genco_rss/rss.py

```
"""Robust secure set (RSS) of the dissertation on PGLib IEEE-118.

For hour h the RSS limits of the index alpha = sum of the non-slack outputs
are the optimal values of two bilevel problems (nature = leader picks the
PV/load vector p in the hourly box, dispatch = follower):

    alpha_max = min_p max_u alpha        alpha_min = max_p min_u alpha

solved with the two-phase Benders decomposition (benders.py) or, for
reference, with the big-M KKT MILP of RSS_MILP2.m (bigm_milp below).
"""
from __future__ import annotations

import time
from dataclasses import dataclass

import numpy as np
import scipy.sparse as sp
from scipy.optimize import Bounds, LinearConstraint, linprog, milp

from . import benders
from .grid import Grid


def dc_opf(grid: Grid, p, rate_scale=None):
    """Linear-cost DC-OPF at p: base dispatch around which ramp limits apply."""
    pmin, pmax = grid.gen_limits()
    A, B, b = grid.rss_matrices(pmin, pmax, rate_scale)
    gc = grid.case["gencost"][grid.disp]
    cost = gc[:, 5] if gc.shape[1] > 5 else np.ones(len(grid.disp))
    r = linprog(cost, A_ub=B, b_ub=b - A @ p, bounds=[(0, None)] * B.shape[1], method="highs")
    if r.status != 0:
        raise RuntimeError("DC-OPF infeasible")
    return r.x
```

```
@dataclass
class RSS:
    hour: int
    upper: benders.Result
    lower: benders.Result
    u_lb: np.ndarray
    u_ub: np.ndarray
    p_lb: np.ndarray
    p_ub: np.ndarray
    A: object = None
    B: object = None
    b: object = None
    d: object = None

    @property
    def alpha_max(self):
        return -self.upper.obj

    @property
    def alpha_min(self):
        return self.lower.obj


def problem(grid: Grid, hour: int, rate_scale=None, ramp_frac=0.3, bess_p=None):
    """Matrices of the RSS bilevel for one hour. bess_p (optional): power
    rating [p.u.] of a battery at each PV bus; each battery adds two follower
    variables (discharge, charge) >= 0 with upper bound bess_p, placed after
    the generators in u (not part of the RSS index)."""
    p_lb, p_ub, p_mid = grid.scenario_box(hour)
    p0 = dc_opf(grid, p_mid)
    u_lb, u_ub = grid.gen_limits(ramp_frac, p0)
    A, B, b = grid.rss_matrices(u_lb, u_ub, rate_scale)
    d = np.ones(B.shape[1]); d[grid.slack_gen] = 0.0
    if bess_p is not None:
        A, B, b, u_ub = _add_bess(grid, A, B, b, u_ub, np.asarray(bess_p, float), rate_scale)
        d = np.r_[d, np.zeros(2 * len(bess_p))]
    return A, B, b, d, p_lb, p_ub, u_lb, u_ub


def _add_bess(grid, A, B, b, u_ub, bp, rate_scale):
    """Append battery columns (discharge +, charge -) at the PV buses."""
    ns, nl, nb_ = len(grid.H), len(grid.rate), len(bp)
    col = np.vstack([Hs[:, grid.pv_bus] for Hs in grid.H])            # line rows, all states
    Bb = np.vstack([np.hstack([col, -col]), np.hstack([-col, col]),
                    np.r_[np.ones(nb_), -np.ones(nb_)][None, :], np.r_[-np.ones(nb_), np.ones(nb_)][None, :],
                    np.zeros((B.shape[0] - 2 * ns * nl - 2, 2 * nb_))])
    B2 = sp.hstack([B, sp.csr_matrix(Bb)])
    B2 = sp.vstack([B2, sp.hstack([sp.csr_matrix((2 * nb_, B.shape[1])), sp.identity(2 * nb_)])]).tocsr()
    A2 = sp.vstack([A, sp.csr_matrix((2 * nb_, A.shape[1]))]).tocsr()
    return A2, B2, np.r_[b, bp, bp], np.r_[u_ub, bp, bp]


def solve_rss(grid: Grid, hour: int, rate_scale=None, ramp_frac=0.3, opts=None) -> RSS:
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = problem(grid, hour, rate_scale, ramp_frac)
    c1 = np.zeros(A.shape[1])
    up = benders.solve(A, B, b, c1, -d, d, p_lb, p_ub, u_ub, opts)
    lo = benders.solve(A, B, b, c1, d, -d, p_lb, p_ub, u_ub, opts)
    return RSS(hour, up, lo, u_lb, u_ub, p_lb, p_ub, A, B, b, d)


def bigm_milp(A, B, b, c1, d1, d2, plb, pub, uub, L=5e4, time_limit=600):
    """KKT + big-M MILP exactly as RSS_MILP2.m (reference). Returns
    (reported objective, true objective of its p, seconds)."""
    A = sp.csr_matrix(A); B = sp.csr_matrix(B)
    m, n_p = A.shape; nu = B.shape[1]; nM = n_p + nu + m
    M = sp.bmat([[sp.csr_matrix((n_p, nM))], [sp.hstack([sp.csr_matrix((nu, n_p + nu)), B.T])],
                 [sp.hstack([-A, -B, sp.csr_matrix((m, m))])]]).tocsr()
    q = np.r_[np.zeros(n_p), -d2, b]; I = sp.identity(nM, format="csr")
    Aall = sp.vstack([sp.hstack([A, B, sp.csr_matrix((m, m + nM))]), sp.hstack([I, -L * I]),
                      sp.hstack([-M, sp.csr_matrix((nM, nM))]), sp.hstack([M, L * I])]).tocsr()
    t = time.time()
    r = milp(-np.r_[c1, d1, np.zeros(m + nM)],
             constraints=LinearConstraint(Aall, -np.inf, np.r_[b, np.zeros(nM), q, L - q]),
             integrality=np.r_[np.zeros(nM), np.ones(nM)],
```

```
            bounds=Bounds(np.r_[plb, np.zeros(nu + m + nM)], np.r_[pub, uub, np.full(m, np.inf),
np.ones(nM)]),
            options={"time_limit": time_limit})
    sec = time.time() - t
    if r.x is None:
        return np.nan, np.nan, sec
    F, _, ok = benders.follower_response(A, B, b, c1, d1, d2, r.x[:n_p])
    return -r.fun, F, sec
```

genco_rss/vsetpoints.py

```
"""Generator voltage set-points per hour from an AC-OPF (pandapower).

gridfm-datakit normally takes generator voltage set-points from an AC-OPF.
PGLib case118 ships flat 1.0 p.u. set-points, which makes reactive limits
meaningless (many units would sit outside their Q range in every case). We
therefore solve one AC-OPF per hour at the centre of the PV/load box and keep
its generator-bus voltages as the set-points of that hour.
"""
from __future__ import annotations

import json
import os
import warnings

import numpy as np

from .grid import DATA, Grid

CACHE = os.path.join(DATA, "vsetpoints.json")


def _opf_vm(grid: Grid, p):
    import pandapower as pp
    from pandapower.converter.pypower import from_ppc

    c = grid.case
    bus = c["bus"].copy()
    pd = bus[:, 2].copy(); qd = bus[:, 3].copy()
    ratio = np.divide(qd, pd, out=np.zeros_like(qd), where=pd > 0)
    pd[grid.load_bus] = p[grid.npv:] * grid.base
    qd[grid.load_bus] = pd[grid.load_bus] * ratio[grid.load_bus]
    pd[grid.pv_bus] -= p[:grid.npv] * grid.base
    bus[:, 2] = pd; bus[:, 3] = qd
    ppc = {"version": "2", "baseMVA": c["baseMVA"], "bus": bus, "gen": c["gen"],
           "branch": c["branch"], "gencost": c["gencost"]}
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        net = from_ppc(ppc, f_hz=60, validate_conversion=False)
        pp.runopp(net, verbose=False)
    vm = net.res_bus.vm_pu.values
    return {int(grid.bus_ids[i]): float(vm[i]) for i in np.unique(grid.gen_bus)}


def setpoints(grid: Grid, recompute=False):
    """dict hour -> {pglib bus number -> Vm set-point}."""
    if os.path.exists(CACHE) and not recompute:
        raw = json.load(open(CACHE))
        return {int(h): {int(b): v for b, v in d.items()} for h, d in raw.items()}
    out = {}
    for h in range(1, 25):
        _, _, p_mid = grid.scenario_box(h)
        out[h] = _opf_vm(grid, p_mid)
    json.dump({str(h): {str(b): v for b, v in d.items()} for h, d in out.items()}, open(CACHE, "w"),
indent=1)
    return out


def gen_vm(grid: Grid, sp_hour: dict) -> np.ndarray:
    """Vm set-point (AC-OPF bus voltage) for every row of the generator table."""
    return np.array([sp_hour[int(grid.bus_ids[b])] for b in grid.gen_bus])
```

```
if __name__ == "__main__":
    g = Grid.load()
    s = setpoints(g, recompute=True)
    v = np.array([list(d.values()) for d in s.values()])
    print("set-points computed for", len(s), "hours; range", v.min().round(4), v.max().round(4))
```

experiments/e1_dc_rss.py

```
"""E1 - certified DC robust secure set, 24 h x 2 limits:
two-phase Benders (this work) vs the big-M KKT MILP of RSS_MILP2.m (L = 5e4).
For the MILP we report its objective AND the true bilevel value of its
leader decision (follower LP re-solved), which exposes big-M errors."""
import os, sys, time
import numpy as np, pandas as pd
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from genco_rss.grid import Grid
from genco_rss import rss, benders

OUT = os.path.join(os.path.dirname(__file__), "..", "results")
g = Grid.load()
rows = []
for h in range(1, 25):
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(g, h)
    c1 = np.zeros(A.shape[1])
    for limit, d1, d2 in (("upper", -d, d), ("lower", d, -d)):
        t0 = time.time(); r = benders.solve(A, B, b, c1, d1, d2, p_lb, p_ub, u_ub); tb = time.time() - t0
        rep, true, tm = rss.bigm_milp(A, B, b, c1, d1, d2, p_lb, p_ub, u_ub)
        sgn = -1 if limit == "upper" else 1
        rows.append(dict(hour=h, limit=limit, alpha=sgn * r.obj, bound=sgn * r.bound, status=r.status,
                         phase2=r.phase2, iters=r.iters, cuts=r.cuts, benders_s=tb,
                         milp_reported=sgn * rep, milp_true=sgn * true, milp_s=tm,
                         n_binary_milp=A.shape[1] + B.shape[1] + A.shape[0]))
        print(f"h{h:02d} {limit:5s} Benders {sgn*r.obj:9.4f} [{r.status}, {r.iters} it, {tb:5.2f}s] | "
              f"MILP reported {sgn*rep:9.4f} true {sgn*true:9.4f} ({tm:6.2f}s)", flush=True)
df = pd.DataFrame(rows)
df.to_csv(os.path.join(OUT, "e1_dc_rss.csv"), index=False)
print(df.groupby("limit")[["benders_s", "milp_s"]].mean())
```

experiments/e3_genco_probe_conformal.py

```
"""E3 - GENCO accuracy, physics-residual probe, residual-normalised conformal screening.

Data (all exact AC by gridfm-datakit + lightsim2grid):
  * iid      : 2,000 draws from the training distribution (seed 1, unseen by training)
  * screen   : 3,000 i.i.d. draws from the RSS screening distribution
               (DC follower responses at Benders / random PV-load vertices),
               split 1,000 tune / 1,000 calibration / 1,000 test
  * unseen   : 600 screening draws in a topology never used in training
               (one circuit of the double corridor 49-66 lost)

Outputs results/e3_{iid,screen,unseen}.parquet and results/e3_summary.json:
accuracy, probe of the corrective-layer residual trajectory (incl. a feature
ablation of physics-trajectory recalibration), conformal screening for three
methods x three alphas, and the miss/flag distribution over 20 random splits.
Set E3_REUSE=1 to reuse the parquet files (skips GENCO inference).
"""
import json, os, sys, time
import numpy as np, pandas as pd
from scipy.stats import spearmanr
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from genco_rss.grid import Grid
from genco_rss import acrss, datagen, rss, benders
from genco_rss.conformal import Screen, true_insecure, trajectory_features, METHODS
from genco_rss.genco import load_latest

HERE = os.path.dirname(__file__)
OUT = os.path.join(HERE, "..", "results")
RUN = os.environ.get("GENCO_RUN", "/root/work/mlruns")
```

```
CFG = os.environ.get("GENCO_CONFIG", os.path.join(HERE, "..", "configs", "genco_pf_case118.yaml"))
WORK = os.environ.get("WORK", "/root/work/eval")
os.makedirs(WORK, exist_ok=True)
g = Grid.load()


def exact(name, items):
    raw_dir = os.path.join(WORK, name)
    raw = os.path.join(raw_dir, "case118_ieee", "raw")
    if os.path.exists(os.path.join(raw, "meta.npz")):
        return acrss.read_ac(raw)
    return acrss.exact_ac(g, items, raw_dir)


def build(name, items):
    tab, L, root, t_ac = exact(name, items)
    t0 = time.time()
    pred = genco.evaluate(os.path.join(root, "case118_ieee"))
    t_eval = time.time() - t0
    df = tab.join(pred.set_index("scenario").drop(columns=[c for c in ["load_true", "vlow_true",
"vhigh_true", "qex_true"] if c in pred.columns]), how="inner")
    df.attrs.update(n=len(df), ac_solver_s=t_ac, genco_model_s=pred.attrs["model_seconds"],
genco_total_s=t_eval)
    df.to_parquet(os.path.join(OUT, f"e3_{name}.parquet"))
    return df


# ---------------------------------------------------------------- data sets
rng = np.random.default_rng(1)
src = datagen.RandomRSS(g, seed=1)
iid_items = [src() for _ in range(2000)]
worst = {}
for h in range(1, 25):
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(g, h)
    for limit, d1, d2 in (("upper", -d, d), ("lower", d, -d)):
        r = benders.solve(A, B, b, np.zeros(A.shape[1]), d1, d2, p_lb, p_ub, u_ub)
        if r.p is not None:
            worst[(h, limit)] = r.p
scr_items = acrss.screening_items(g, 3000, seed=2, worst=worst)
uns_items = [(h, 3, p, u) for (h, s, p, u) in acrss.screening_items(g, 600, seed=3, worst=worst)]

if os.environ.get("E3_DATA_ONLY"):
    for n, it in (("iid", iid_items), ("screen", scr_items), ("unseen", uns_items)):
        exact(n, it)
    sys.exit(0)
if os.environ.get("E3_REUSE") and os.path.exists(os.path.join(OUT, "e3_screen.parquet")):
    rp = lambda n: pd.read_parquet(os.path.join(OUT, f"e3_{n}.parquet"))
    iid, scr, uns = rp("iid"), rp("screen"), rp("unseen")
    S0 = json.load(open(os.path.join(OUT, "e3_summary.json")))
    n_params, acc0 = S0["genco_params"], S0["accuracy"]
else:
    genco = load_latest(RUN, CFG)
    n_params, acc0 = genco.n_params, None
    print("GENCO parameters:", n_params)
    iid = build("iid", iid_items)
    scr = build("screen", scr_items)
    uns = build("unseen", uns_items)


# ---------------------------------------------------------------- accuracy
def acc(df):
    e = df.load_pred - df.load_true
    return {"n": len(df), "load_MAE": float(e.abs().mean()), "load_p95_abs_err":
float(e.abs().quantile(0.95)),
            "load_bias": float(e.mean()),
            "vm_err_max_mean": float(df.vm_err_max.mean()), "pbe_mean_MVA": float(df.pbe_mean.mean()),
            "insecure_frac": float(true_insecure(df).mean()),
            "genco_ms_per_scenario": 1e3 * df.attrs["genco_model_s"] / len(df),
            "ac_ms_per_scenario": 1e3 * df.attrs["ac_solver_s"] / len(df)}


if acc0 is None:
    accuracy = {k: acc(v) for k, v in [("iid", iid), ("screen", scr), ("unseen_topology", uns)]}
else:   # parquet attrs are not kept: reuse timings from the previous summary
    accuracy = acc0
summary = {"genco_params": int(n_params), "accuracy": accuracy}
```

```
# ------------------------------------------------------------------ probe
from sklearn.metrics import roc_auc_score
from sklearn.ensemble import GradientBoostingRegressor


def probe(df):
    err = (df.load_pred - df.load_true).abs()
    big = err > err.quantile(0.9)
    return {q: {"spearman_vs_abs_load_error": float(spearmanr(df[q], err).correlation),
                "auroc_top10pct_error": float(roc_auc_score(big, df[q]))}
            for q in ["r_last", "rho", "pbe_mean", "r_first"]}


summary["probe"] = {"screen": probe(scr), "iid": probe(iid)}
both = pd.concat([scr.assign(unseen=0), uns.assign(unseen=1)])
summary["probe"]["ood_detection_auroc"] = {q: float(roc_auc_score(both.unseen, both[q])) for q in ["r_last",
"rho", "pbe_mean"]}
summary["probe"]["r_last_median"] = {"screen": float(scr.r_last.median()), "unseen":
float(uns.r_last.median()),
                                     "iid": float(iid.r_last.median())}

perm = np.random.default_rng(0).permutation(len(scr))
tune, cal, test = scr.iloc[perm[:1000]], scr.iloc[perm[1000:2000]], scr.iloc[perm[2000:]]

# feature ablation of physics-trajectory recalibration (fit on tune, evaluate on test / unseen)
F_all = trajectory_features(scr)
RK = [c for c in F_all if c[0] == "r" and c[1:].isdigit()]
ablation = {"none (raw GENCO)": None, "GENCO load prediction": ["load_pred"],
            "GENCO outputs": [c for c in F_all if c.endswith("_pred")],
            "residual trajectory only": RK + ["rho"],
            "load prediction + residual trajectory": ["load_pred"] + RK + ["rho"],
            "GENCO outputs + residual trajectory (PTR)": F_all}
abl = {}
for name, F in ablation.items():
    row = {}
    for tag, d in (("test", test), ("unseen_topology", uns)):
        row[f"{tag}_MAE"] = float((d.load_true - d.load_pred).abs().mean()) if F is None else None
    if F is not None:
        m = GradientBoostingRegressor(n_estimators=300, max_depth=3, learning_rate=0.05, subsample=0.8,
                                      random_state=0).fit(tune[F].values, (tune.load_true -
tune.load_pred).values)
        for tag, d in (("test", test), ("unseen_topology", uns)):
            row[f"{tag}_MAE"] = float((d.load_true - d.load_pred - m.predict(d[F].values)).abs().mean())
    if F is not None and len(F) > 1:
        S = Screen(alpha=0.05, method="ptr", features=F).fit(tune, cal)
    else:
        S = Screen(alpha=0.05, method="plain").fit(tune, cal) if F is None else Screen(alpha=0.05,
method="ptr", features=F).fit(tune, cal)
    row.update({f"{tag}_{x}": S.report(d)[x] for tag, d in (("test", test), ("unseen_topology", uns))
                for x in ("flag_rate", "missed", "secure_cleared_frac")})
    abl[name] = row
summary["probe"]["trajectory_ablation"] = abl
pd.DataFrame(abl).T.to_csv(os.path.join(OUT, "e3_ptr_ablation.csv"))

# ------------------------------------------------------------------ conformal (fixed split)
res = {}
for alpha in [0.01, 0.05, 0.10]:
    for method in METHODS:
        S = Screen(alpha=alpha, method=method).fit(tune, cal)
        res[f"alpha={alpha} {method}"] = {"test": S.report(test), "unseen_topology": S.report(uns),
                                          "c": {k: float(v) for k, v in S.c.items()}}
summary["conformal"] = res

# ------------------------------------------------------------------ 20 random splits
rows = []
for rep in range(20):
    pm = np.random.default_rng(100 + rep).permutation(len(scr))
    tu, ca, te = scr.iloc[pm[:1000]], scr.iloc[pm[1000:2000]], scr.iloc[pm[2000:]]
    for alpha in [0.01, 0.05, 0.10]:
        for method in METHODS:
            S = Screen(alpha=alpha, method=method).fit(tu, ca, seed=rep)
            for tag, d in (("test", te), ("unseen_topology", uns)):
                r = S.report(d)
                rows.append({"rep": rep, "alpha": alpha, "method": method, "set": tag, **r})
reps = pd.DataFrame(rows)
```

```
reps.to_csv(os.path.join(OUT, "e3_conformal_splits.csv"), index=False)
summary["conformal_splits"] = (reps.groupby(["set", "alpha", "method"])[["flag_rate", "miss_rate",
"secure_cleared_frac"]]
                               .agg(["mean", "max"]).round(5).reset_index()
                               .pipe(lambda t: {f"{a} {b} {c}": {f"{x}_{y}": t.loc[i, (x, y)]
                                                                 for x in ("flag_rate", "miss_rate",
"secure_cleared_frac")
                                                                 for y in ("mean", "max")}
                                                for i, (a, b, c) in enumerate(zip(t["set"], t["alpha"],
t["method"]))}))
json.dump(summary, open(os.path.join(OUT, "e3_summary.json"), "w"), indent=1, default=float)
print(json.dumps({k: summary[k] for k in ("accuracy", "conformal_splits")}, indent=1, default=float))
print(pd.DataFrame(abl).T.round(4).to_string())
```

experiments/e4_ac_rss.py

```
"""E4 - AC-aware robust secure set, 24 h x 2 limits (DC Benders + GENCO
screening + GENCO-gradient adversary + exact AC verification and cuts).
The conformal screen is re-fitted from the E3 tune/calibration data."""
import json, os, sys, time
import numpy as np, pandas as pd
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from genco_rss.grid import Grid
from genco_rss import acrss
from genco_rss.conformal import Screen
from genco_rss.genco import load_latest

HERE = os.path.dirname(__file__)
OUT = os.path.join(HERE, "..", "results")
RUN = os.environ.get("GENCO_RUN", "/root/work/mlruns")
CFG = os.environ.get("GENCO_CONFIG", os.path.join(HERE, "..", "configs", "genco_pf_case118.yaml"))
ALPHA = float(os.environ.get("ALPHA", "0.05"))
HOURS = [int(h) for h in os.environ.get("HOURS", ",".join(map(str, range(1, 25)))).split(",")]

g = Grid.load()
genco = load_latest(RUN, CFG)
scr = pd.read_parquet(os.path.join(OUT, "e3_screen.parquet"))
perm = np.random.default_rng(0).permutation(len(scr))
screen = Screen(alpha=ALPHA, method=os.environ.get("SCREEN", "ptr")).fit(scr.iloc[perm[:1000]],
scr.iloc[perm[1000:2000]])
print("screen thresholds", screen.c)

rows, hist = [], []
for h in HOURS:
    for limit in ("upper", "lower"):
        t0 = time.time()
        r = acrss.ac_rss(g, h, limit, genco=genco, screen=screen)
        r["seconds"] = time.time() - t0
        hist += r.pop("history")
        rows.append(r)
        print(f"h{h:02d} {limit:5s}: alpha DC {r['alpha_dc']:.4f} -> AC {r['alpha_ac']:.4f}  "
              f"({r['iters']} it, {r['n_derated']} derated, {r['seconds']:.0f}s)", flush=True)
        pd.DataFrame(rows).to_csv(os.path.join(OUT, "e4_ac_rss.csv"), index=False)
        pd.DataFrame(hist).to_csv(os.path.join(OUT, "e4_ac_rss_iterations.csv"), index=False)
```

experiments/e5_bess.py

```
"""E5 - extension: second-life EV batteries at the 20 PV buses as dispatch
(follower) resources in the RSS bilevel, 24 h x 2 limits, solved with the
two-phase Benders decomposition. The worst-case operating points (p*, u*)
with and without batteries are verified with exact AC in the 3 topologies."""
import os, sys
import numpy as np, pandas as pd
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from genco_rss.grid import Grid
from genco_rss import rss, benders, acrss
from genco_rss.bess import bess_power

OUT = os.path.join(os.path.dirname(__file__), "..", "results")
```

```
g = Grid.load()
bp = bess_power(g)
rows, items, keys = [], [], []
for h in range(1, 25):
    for case, bpx in (("no_bess", None), ("bess", bp)):
        A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(g, h, bess_p=bpx)
        c1 = np.zeros(A.shape[1])
        for limit, d1, d2 in (("upper", -d, d), ("lower", d, -d)):
            r = benders.solve(A, B, b, c1, d1, d2, p_lb, p_ub, u_ub)
            sgn = -1 if limit == "upper" else 1
            nb = len(bp) if bpx is not None else 0
            net_bess = float((r.u[-2 * nb:-nb] - r.u[-nb:]).sum()) if nb else 0.0
            rows.append(dict(hour=h, case=case, limit=limit, alpha=sgn * r.obj, bound=sgn * r.bound,
                             status=r.status, seconds=r.time, bess_net_injection=net_bess))
            for s in range(3):
                items.append((h, s, r.p, r.u)); keys.append((h, case, limit, s))
            print(f"h{h:02d} {case:7s} {limit:5s} alpha={sgn*r.obj:8.4f} [{r.status}] bess_net=
{net_bess:+.3f}", flush=True)
df = pd.DataFrame(rows)
tab, L, root, t_ac = acrss.exact_ac(g, items, "/root/work/eval/bess")
k = pd.DataFrame(keys, columns=["hour", "case", "limit", "state"])
k["load_true"] = tab.load_true.values
ac = k.groupby(["hour", "case", "limit"]).load_true.max().rename("ac_max_loading_worst_case").reset_index()
df = df.merge(ac, on=["hour", "case", "limit"])
df.to_csv(os.path.join(OUT, "e5_bess.csv"), index=False)
w = df.pivot_table(index=["hour", "case"], columns="limit", values="alpha").reset_index()
w["d_rss"] = w["upper"] - w["lower"]
w = w.pivot(index="hour", columns="case", values="d_rss")
w["gain"] = w["bess"] - w["no_bess"]
w["gain_over_2Pbess"] = w["gain"] / (2 * bp.sum())
w.to_csv(os.path.join(OUT, "e5_bess_width.csv"))
print(w.round(3))
print("battery fleet P_eff total [p.u.]:", round(float(bp.sum()), 3))
```

experiments/e6_figures.py

```
"""Figures for the post and the paper (all from results/*)."""
import json, os
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

R = os.path.join(os.path.dirname(__file__), "..", "results")
F = os.path.join(R, "figures"); os.makedirs(F, exist_ok=True)
C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100"]
GRAY = "#8a8984"
plt.rcParams.update({"font.size": 9, "axes.titlesize": 10, "axes.titleweight": "bold",
                     "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                     "grid.alpha": 0.25, "grid.linewidth": 0.6, "lines.linewidth": 2, "legend.frameon":
False})


def rd(n):
    p = os.path.join(R, n)
    if not os.path.exists(p):
        return None
    return pd.read_parquet(p) if p.endswith(".parquet") else pd.read_csv(p)


# ---------------------------------------------------------------- Fig. 1 DC
e1, e4 = rd("e1_dc_rss.csv"), rd("e4_ac_rss.csv")
if e1 is not None:
    fig, ax = plt.subplots(1, 3, figsize=(13, 3.6), dpi=200)
    up, lo = e1[e1.limit == "upper"].set_index("hour"), e1[e1.limit == "lower"].set_index("hour")
    ax[0].fill_between(up.index, lo.alpha, up.alpha, color=C[0], alpha=0.15, lw=0)
    ax[0].plot(up.index, up.alpha, color=C[0], label="DC limits (Benders, certified)")
    ax[0].plot(lo.index, lo.alpha, color=C[0])
    if e4 is not None:
        u4, l4 = e4[e4.limit == "upper"].set_index("hour"), e4[e4.limit == "lower"].set_index("hour")
        ax[0].plot(u4.index, u4.alpha_ac, color=C[1], ls="--", label="AC-corrected limits")
        ax[0].plot(l4.index, l4.alpha_ac, color=C[1], ls="--")
```

```
        ax[0].fill_between(u4.index, l4.alpha_ac, u4.alpha_ac, where=(u4.alpha_ac >= l4.alpha_ac).values,
                           color=C[1], alpha=0.18, lw=0, interpolate=True)
        emp = u4.index[(u4.alpha_ac < l4.alpha_ac).values]
        if len(emp):
            ax[0].scatter(emp, l4.alpha_ac[emp], marker="x", color="k", s=30, zorder=5, label="AC-corrected
set empty")
    ax[0].set(title="(a) Robust secure range per hour", xlabel="hour", ylabel="RSS index [p.u., 100 MVA]")
    ax[0].legend(fontsize=8)
    for i, lim in enumerate(("lower", "upper")):
        d = e1[e1.limit == lim]
        ax[1].bar(d.hour + (i - 0.5) * 0.4, (d.milp_reported - d.milp_true).abs() * 100, width=0.4,
                  color=C[i], label=f"{lim} limit")
    ax[1].set(title="(b) big-M MILP: reported minus attained", xlabel="hour", ylabel="|reporting error|
[MW]")
    ax[1].legend(fontsize=8)
    x = np.arange(1, 25)
    for i, (lab, col, c) in enumerate((("big-M KKT MILP", GRAY, "milp_s"), ("two-phase Benders", C[0],
"benders_s"))):
        v = e1.groupby("hour")[c].sum()
        ax[2].plot(x, v.values, color=col, marker="o", ms=3, label=lab)
    ax[2].set_yscale("log"); ax[2].set(title="(c) Measured time per hour (both limits)", xlabel="hour",
ylabel="seconds")
    ax[2].legend(fontsize=8)
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig1_dc_rss.png")); plt.close(fig)

# ---------------------------------------------------------------- Fig. 2 GENCO probe
scr, uns, iid = rd("e3_screen.parquet"), rd("e3_unseen.parquet"), rd("e3_iid.parquet")
if scr is not None:
    fig, ax = plt.subplots(1, 3, figsize=(13, 3.8), dpi=200)
    names = ["base", "69-70", "100-103"]
    for s in range(3):
        d = scr[scr.state == s]
        ax[0].scatter(d.load_true, d.load_pred, s=5, alpha=0.5, color=C[s], label=names[s], linewidths=0)
    lim = [min(scr.load_true.min(), scr.load_pred.min()), max(scr.load_true.max(), scr.load_pred.max())]
    ax[0].plot(lim, lim, color=GRAY, lw=1)
    ax[0].axvline(1, color=GRAY, lw=0.8, ls=":"); ax[0].axhline(1, color=GRAY, lw=0.8, ls=":")
    ax[0].set(title="(a) GENCO vs exact AC: max branch loading", xlabel="exact AC", ylabel="GENCO")
    ax[0].legend(fontsize=8, markerscale=3)
    rcols = sorted([c for c in scr.columns if c[0] == "r" and c[1:].isdigit()], key=lambda c: int(c[1:]))
    L = np.arange(1, len(rcols) + 1)
    for d, lab, col in ((scr, "seen topologies", C[0]), (uns, "unseen topology (49-66)", C[1])):
        if d is None:
            continue
        q = d[rcols].quantile([0.1, 0.5, 0.9]).values
        ax[1].plot(L, q[1], color=col, marker="o", ms=3, label=lab)
        ax[1].fill_between(L, q[0], q[2], color=col, alpha=0.15, lw=0)
    ax[1].set_yscale("log")
    ax[1].set(title="(b) Mismatch inside GENCO, layer by layer", xlabel="corrective layer l",
ylabel="||r(l)|| (median, 10-90 %)")
    ax[1].legend(fontsize=8)
    ab = rd("e3_ptr_ablation.csv")
    if ab is not None:
        ab = ab.set_index(ab.columns[0])
        lab = {"none (raw GENCO)": "raw GENCO", "GENCO load prediction": "+ g(load pred.)",
               "GENCO outputs": "+ g(outputs)", "residual trajectory only": "+ g(r(1..K))",
               "load prediction + residual trajectory": "+ g(load, r(1..K))",
               "GENCO outputs + residual trajectory (PTR)": "PTR: + g(outputs, r(1..K))"}
        y = np.arange(len(ab))
        ax[2].barh(y + 0.2, ab.test_MAE * 100, height=0.38, color=C[0], label="seen topologies (test)")
        ax[2].barh(y - 0.2, ab.unseen_topology_MAE * 100, height=0.38, color=C[1], label="unseen topology")
        ax[2].set_yticks(y, [lab.get(i, i) for i in ab.index], fontsize=7); ax[2].invert_yaxis()
        ax[2].set(title="(c) Recalibration from GENCO's trajectory", xlabel="max-loading MAE [% of rating]",
                  xlim=(0, 1.45 * ab.test_MAE.max() * 100))
        ax[2].legend(fontsize=7, loc="lower right")
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig2_genco_probe.png")); plt.close(fig)

# ---------------------------------------------------------------- Fig. 3 conformal (20 random splits)
cs = rd("e3_conformal_splits.csv")
if cs is not None:
    fig, ax = plt.subplots(1, 3, figsize=(13, 3.4), dpi=200)
    names = {"plain": "plain split-conformal", "normalised": "residual-normalised", "ptr": "PTR (this work)"}
    for i, m in enumerate(("plain", "normalised", "ptr")):
        for j, (st, ttl) in enumerate((("test", "seen topologies"), ("unseen_topology", "unseen topology 49-
66"))):
            g = cs[(cs.method == m) & (cs.set == st)].groupby("alpha").secure_cleared_frac
            ax[j].plot(g.mean().index, g.mean().values * 100, marker="o", ms=4, color=C[i], label=names[m])
```

```
            ax[j].fill_between(g.mean().index, g.min().values * 100, g.max().values * 100, color=C[i],
alpha=0.15, lw=0)
            ax[j].set(title=f"({'ab'[j]}) Secure scenarios cleared, {ttl}", xlabel="alpha",
                      ylabel="exact AC avoided [% of secure]")
        g = cs[(cs.method == m) & (cs.set == "test")].groupby("alpha").miss_rate
        ax[2].plot(g.mean().index, g.mean().values, marker="o", ms=4, color=C[i], label=names[m])
        ax[2].fill_between(g.mean().index, g.min().values, g.max().values, color=C[i], alpha=0.15, lw=0)
    al = np.array([0.01, 0.05, 0.10])
    ax[2].plot(al, al, color=GRAY, lw=1, ls="--", label="guarantee: miss <= alpha")
    ax[2].set(title="(c) Missed insecure scenarios (seen)", xlabel="alpha", ylabel="P(insecure and not
flagged)")
    ax[0].legend(fontsize=7); ax[2].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig3_conformal.png")); plt.close(fig)

# ---------------------------------------------------------------- Fig. 6 PTR screen at alpha = 0.05
if scr is not None:
    import sys
    sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
    from genco_rss.conformal import Screen
    perm = np.random.default_rng(0).permutation(len(scr))
    tune, cal, test = scr.iloc[perm[:1000]], scr.iloc[perm[1000:2000]], scr.iloc[perm[2000:]]
    fig, ax = plt.subplots(1, 3, figsize=(13, 3.6), dpi=200)
    for i, (m, ttl) in enumerate((("plain", "(a) plain split-conformal"), ("ptr", "(b) PTR, seen
topologies"), ("ptr", "(c) PTR, unseen topology 49-66"))):
        S = Screen(alpha=0.05, method=m).fit(tune, cal)
        d = uns if i == 2 else test
        f = S.flag(d); ub = S.upper(d, "load")
        ax[i].scatter(d.load_true[f], ub[f], s=5, color=GRAY, alpha=0.45, linewidths=0, label=f"sent to exact
AC ({f.mean():.0%})")
        ax[i].scatter(d.load_true[~f], ub[~f], s=6, color=C[2], linewidths=0, label=f"cleared
({(~f).mean():.0%})")
        miss = (~f) & (d.load_true.values > 1)
        if miss.any():
            ax[i].scatter(d.load_true[miss], ub[miss], s=30, facecolors="none", edgecolors=C[1],
label=f"missed ({miss.sum()})")
        ax[i].axvline(1, color="k", lw=0.7, ls=":"); ax[i].axhline(1, color="k", lw=0.7, ls=":")
        ax[i].set(title=ttl, xlabel="exact AC max branch loading", ylabel="conformal upper bound (alpha =
0.05)")
        ax[i].legend(fontsize=7, loc="upper left", markerscale=2)
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig6_screen.png")); plt.close(fig)

# ---------------------------------------------------------------- Fig. 4 AC loop
it = rd("e4_ac_rss_iterations.csv")
if e4 is not None and it is not None:
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.4), dpi=200)
    for i, lim in enumerate(("upper", "lower")):
        d = e4[e4.limit == lim]
        sh = (d.alpha_dc - d.alpha_ac) if lim == "upper" else (d.alpha_ac - d.alpha_dc)
        ax[0].bar(d.hour + (i - 0.5) * 0.4, sh * 100, width=0.4, color=C[i], label=f"{lim} limit")
    ax[0].set(title="(a) Secure range removed by the AC correction", xlabel="hour", ylabel="limit moved
inward [MW]")
    ax[0].legend(fontsize=8)
    f = it[it.iter == 1].dropna(subset=["max_load_adversarial"])
    ax[1].scatter(f.max_load_random, f.max_load_adversarial, s=14, color=C[0])
    m = [min(f.max_load_random.min(), f.max_load_adversarial.min()), max(f.max_load_random.max(),
f.max_load_adversarial.max())]
    ax[1].plot(m, m, color=GRAY, lw=1)
    ax[1].set(title="(b) GENCO-gradient adversary vs random vertices", xlabel="worst exact-AC loading, random
+ Benders vertices",
              ylabel="worst exact-AC loading, adversarial")
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig4_ac_loop.png")); plt.close(fig)

# ---------------------------------------------------------------- Fig. 5 batteries
w = rd("e5_bess_width.csv")
if w is not None:
    fig, ax = plt.subplots(figsize=(6, 3.2), dpi=200)
    ax.plot(w.hour, w.no_bess, color=C[0], marker="o", ms=3, label="without batteries")
    ax.plot(w.hour, w.bess, color=C[2], marker="o", ms=3, label="with second-life batteries")
    ax.set(title="RSS width (upper - lower limit)", xlabel="hour", ylabel="[p.u., 100 MVA]")
    ax.legend(fontsize=8)
    fig.tight_layout(); fig.savefig(os.path.join(F, "fig5_bess.png")); plt.close(fig)
print("figures in", F, sorted(os.listdir(F)))
```

experiments/e7_numbers.py

```
"""Collect every number quoted in the post, README and paper from results/*
into results/numbers.json and paper/numbers.tex (single source of truth)."""
import json, os
import numpy as np, pandas as pd

HERE = os.path.dirname(__file__)
R = os.path.join(HERE, "..", "results")
N = {}


def rd(n):
    p = os.path.join(R, n)
    return pd.read_csv(p) if os.path.exists(p) else None


e1 = rd("e1_dc_rss.csv")
if e1 is not None:
    err = (e1.milp_reported - e1.milp_true).abs()
    N.update(e1_problems=len(e1), benders_mean_s=e1.benders_s.mean(), benders_max_s=e1.benders_s.max(),
             milp_mean_s=e1.milp_s.mean(), milp_max_s=e1.milp_s.max(),
             speedup_mean=e1.milp_s.sum() / e1.benders_s.sum(),
             benders_one_iter=int((e1.iters == 1).sum()), benders_phase2=int(e1.phase2.sum()),
             benders_certified=int(e1.status.isin(["optimal", "gap"]).sum()),
             benders_gap_max=float(((e1.bound - e1.alpha).abs() / e1.bound.abs().clip(lower=1)).max()),
             bigm_err_n=int((err > 1e-4).sum()), bigm_err_max=float(err.max()),
bigm_err_max_MW=float(err.max() * 100),
             bigm_subopt_n=int(((e1.alpha - e1.milp_true).abs() > 1e-4).sum()),
             bigm_err_hour=int(e1.loc[err.idxmax(), "hour"]), bigm_err_limit=str(e1.loc[err.idxmax(),
"limit"]),
             benders_vs_milp_true_maxdiff=float((e1.alpha - e1.milp_true).abs().max()),
             dc_width_mean_MW=float((e1[e1.limit == "upper"].set_index("hour").alpha
                                     - e1[e1.limit == "lower"].set_index("hour").alpha).mean() * 100))

sp = os.path.join(R, "e3_summary.json")
if os.path.exists(sp):
    S = json.load(open(sp))
    a = S["accuracy"]
    N.update(genco_params=S["genco_params"], load_mae_iid=a["iid"]["load_MAE"], load_mae_screen=a["screen"]
["load_MAE"],
             load_mae_unseen=a["unseen_topology"]["load_MAE"], load_p95_screen=a["screen"]
["load_p95_abs_err"],
             vm_err_screen=a["screen"]["vm_err_max_mean"], insecure_screen=a["screen"]["insecure_frac"],
             insecure_unseen=a["unseen_topology"]["insecure_frac"],
             genco_ms=a["screen"]["genco_ms_per_scenario"], ac_ms=a["screen"]["ac_ms_per_scenario"])
    pr = S["probe"]
    N.update(sp_rlast=pr["screen"]["r_last"]["spearman_vs_abs_load_error"], auc_rlast=pr["screen"]["r_last"]
["auroc_top10pct_error"],
             sp_rho=pr["screen"]["rho"]["spearman_vs_abs_load_error"], auc_rho=pr["screen"]["rho"]
["auroc_top10pct_error"],
             sp_rfirst=pr["screen"]["r_first"]["spearman_vs_abs_load_error"],
             ood_auc_rlast=pr["ood_detection_auroc"]["r_last"], ood_auc_rho=pr["ood_detection_auroc"]["rho"],
             rlast_med_seen=pr["r_last_median"]["screen"], rlast_med_unseen=pr["r_last_median"]["unseen"])
    pr_ab = pr["trajectory_ablation"]
    N.update(ptr_mae_test=pr_ab["GENCO outputs + residual trajectory (PTR)"]["test_MAE"],
             ptr_mae_unseen=pr_ab["GENCO outputs + residual trajectory (PTR)"]["unseen_topology_MAE"],
             raw_mae_test=pr_ab["none (raw GENCO)"]["test_MAE"], raw_mae_unseen=pr_ab["none (raw GENCO)"]
["unseen_topology_MAE"],
             traj_only_mae_test=pr_ab["residual trajectory only"]["test_MAE"],
             outputs_mae_test=pr_ab["GENCO outputs"]["test_MAE"],
             loadpred_mae_test=pr_ab["GENCO load prediction"]["test_MAE"])
    N["ptr_mae_red_pct"] = 100 * (1 - N["ptr_mae_test"] / N["raw_mae_test"])
    N["traj_only_mae_red_pct"] = 100 * (1 - N["traj_only_mae_test"] / N["raw_mae_test"])
    CS = S["conformal_splits"]
    for st, stag in (("test", ""), ("unseen_topology", "unseen_")):
        for al in ("0.01", "0.05", "0.1"):
            for m in ("plain", "normalised", "ptr"):
                v = CS[f"{st} {al} {m}"]; sfx = f"{stag}{m}_{al.replace('.', '')}"
                N[f"clear_{sfx}"] = 100 * v["secure_cleared_frac_mean"]
                N[f"flag_{sfx}"] = 100 * v["flag_rate_mean"]
                N[f"miss_{sfx}"] = 100 * v["miss_rate_mean"]
                N[f"missmax_{sfx}"] = 100 * v["miss_rate_max"]
```

```
    C = S["conformal"]["alpha=0.05 ptr"]["test"]
    N["test_n"] = C["n"]; N["test_insecure"] = C["insecure"]
    N["n_splits"] = 20

e4, it = rd("e4_ac_rss.csv"), rd("e4_ac_rss_iterations.csv")
if e4 is not None:
    e4["shift_MW"] = (e4.alpha_ac - e4.alpha_dc).abs() * 100
    up, lo = e4[e4.limit == "upper"].set_index("hour"), e4[e4.limit == "lower"].set_index("hour")
    w_dc, w_ac = (up.alpha_dc - lo.alpha_dc) * 100, (up.alpha_ac - lo.alpha_ac) * 100
    k = int(e4.shift_MW.idxmax())
    N.update(e4_limits=len(e4), e4_changed=int((e4.shift_MW > 0.1).sum()),
e4_shift_max_MW=float(e4.shift_MW.max()),
             e4_shift_max_hour=int(e4.loc[k, "hour"]), e4_shift_max_limit=str(e4.loc[k, "limit"]),
             e4_shift_mean_MW=float(e4.shift_MW.mean()), e4_width_dc_mean_MW=float(w_dc.mean()),
             e4_width_ac_mean_MW=float(w_ac.mean()), e4_width_red_mean_pct=float(((w_dc - w_ac) /
w_dc).mean() * 100),
             e4_width_red_max_pct=float(((w_dc - w_ac) / w_dc).max() * 100),
             e4_empty_hours=int((w_ac < 0).sum()), e4_empty_hour_list=",".join(str(h) for h in w_ac[w_ac <
0].index),
             e4_empty_dc_width_MW=float(w_dc[w_ac < 0].iloc[0]) if (w_ac < 0).any() else float('nan'),
             e4_iters_mean=float(e4.iters.mean()), e4_iters_max=int(e4.iters.max()),
             e4_infeasible=int(e4.alpha_ac.isna().sum()), e4_seconds_mean=float(e4.seconds.mean()))
if it is not None:
    f1 = it[it.iter == 1]
    N.update(it1_dc_boundary_insecure=int((f1.max_load_true > 1 + 1e-6).sum()), it1_n=len(f1),
             it1_max_load=float(f1.max_load_true.max()), pool_total=int(it.pool.sum()),
             flagged_total=int(it.flagged.sum()), missed_total=int(it.missed_by_screen.clip(lower=0).sum()),
             genco_s_total=float(it.genco_s.sum()), exact_ac_s_total=float(it.exact_ac_solver_s.sum()),
             pool_cleared_pct=float(100 * (1 - it.flagged.sum() / it.pool.sum())),
             it1_dc_boundary_insecure_pct=float(100 * (f1.max_load_true > 1 + 1e-6).mean()))
    adv = it.dropna(subset=["max_load_adversarial"])
    N.update(adv_cases=len(adv), adv_wins=int((adv.max_load_adversarial > adv.max_load_random + 1e-6).sum()),
             adv_gain_max=float((adv.max_load_adversarial - adv.max_load_random).max()),
             adv_gain_mean=float((adv.max_load_adversarial - adv.max_load_random).mean()))

w = rd("e5_bess_width.csv")
b5 = rd("e5_bess.csv")
if w is not None:
    N.update(bess_gain_mean_MW=float(w.gain.mean() * 100), bess_gain_min_MW=float(w.gain.min() * 100),
             bess_gain_max_MW=float(w.gain.max() * 100), bess_frac_mean=float(w.gain_over_2Pbess.mean()),
             bess_frac_min=float(w.gain_over_2Pbess.min()), width_nobess_mean_MW=float(w.no_bess.mean() *
100))
if b5 is not None:
    N.update(bess_ac_insecure=int((b5[b5.case == "bess"].ac_max_loading_worst_case > 1).sum()),
             nobess_ac_insecure=int((b5[b5.case == "no_bess"].ac_max_loading_worst_case > 1).sum()),
             bess_rows=int((b5.case == "bess").sum()))

N = {k: (float(v) if isinstance(v, (np.floating, float)) else v) for k, v in N.items()}
for k in [k for k in N if ("mae" in k or k.startswith(("it1_max_load", "adv_gain", "load_p95", "insecure_")))
and isinstance(N[k], float) and not k.endswith("_pct")]:
    N[k + "_pct"] = 100 * N[k]
json.dump(N, open(os.path.join(R, "numbers.json"), "w"), indent=1)
os.makedirs(os.path.join(HERE, "..", "paper"), exist_ok=True)
with open(os.path.join(HERE, "..", "paper", "numbers.tex"), "w") as f:
    for k, v in N.items():
        name = "N" + "".join(ch for ch in k.translate(str.maketrans("0123456789", "abcdefghij")).title() if
ch.isalpha())
        if isinstance(v, float):
            v = f"{v:.3g}" if abs(v) < 100 else (f"{v:.1f}" if abs(v) < 1000 else f"{v:,.0f}".replace(",", "
{,}"))
        if isinstance(v, int) and abs(v) >= 1000:
            v = f"{v:,}".replace(",", "{,}")
        f.write(f"\\newcommand{{\\{name}}}{{{v}}}\n")
print(json.dumps(N, indent=1))
```

experiments/e8_docs.py

```
"""Render docs/templates/*.md with results/numbers.json -> docs/*.md
(no number in the post or README is typed by hand)."""
import json, os, string

HERE = os.path.dirname(__file__)
```

```
ROOT = os.path.join(HERE, "..")
N = json.load(open(os.path.join(ROOT, "results", "numbers.json")))

# derived display values
pct = lambda k: 100 * N[k] if k in N else float("nan")
N.update(it1_max_load_pct=pct("it1_max_load"), load_mae_screen_pct=pct("load_mae_screen"),
         load_mae_unseen_pct=pct("load_mae_unseen"), load_mae_iid_pct=pct("load_mae_iid"),
         adv_gain_max_pct=pct("adv_gain_max"), ptr_mae_test_pct=pct("ptr_mae_test"),
         ptr_mae_unseen_pct=pct("ptr_mae_unseen"), raw_mae_test_pct=pct("raw_mae_test"))
links = {"SPACE_URL": os.environ.get("SPACE_URL", "<SPACE_URL>"), "CODE_URL": os.environ.get("CODE_URL", "
<CODE_URL>"),
         "PAPER_URL": os.environ.get("PAPER_URL", "<PAPER_URL>"),
         # for the Community post set
IMG_BASE=https://huggingface.co/spaces/<user>/<space>/resolve/main/results/figures
         "IMG_BASE": os.environ.get("IMG_BASE", "results/figures")}


class Fmt(string.Formatter):
    def get_value(self, key, args, kwargs):
        if key not in kwargs:
            raise KeyError(f"number '{key}' not in results/numbers.json")
        return kwargs[key]


tdir = os.path.join(ROOT, "docs", "templates")
for name in sorted(os.listdir(tdir)):
    src = open(os.path.join(tdir, name)).read()
    for k, v in links.items():
        src = src.replace(f"<{k}>", v)
    # blocks between <!-- raw --> and <!-- /raw --> (LaTeX) are copied verbatim, the rest is formatted
    out, rest = "", src
    while "<!-- raw -->" in rest:
        pre, tail = rest.split("<!-- raw -->", 1)
        raw, rest = tail.split("<!-- /raw -->", 1)
        out += Fmt().vformat(pre, (), N) + raw.strip("\n") + "\n"
    out += Fmt().vformat(rest, (), N)
    dst = os.path.join(ROOT, "README.md" if name == "README_space.md" else os.path.join("docs", name))
    open(dst, "w").write(out)
    print("wrote", os.path.relpath(dst, ROOT))
```

tests/test_core.py

```
"""Fast checks of the two certified components (run: pytest -q tests)."""
import numpy as np
import pandas as pd

from genco_rss import benders, rss
from genco_rss.conformal import Screen, QUANTS
from genco_rss.grid import Grid


def test_benders_matches_true_bilevel_value():
    g = Grid.load()
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(g, 12)
    c1 = np.zeros(A.shape[1])
    for d1, d2 in ((-d, d), (d, -d)):
        r = benders.solve(A, B, b, c1, d1, d2, p_lb, p_ub, u_ub)
        assert r.status == "optimal"
        assert abs(r.obj - r.bound) <= 1e-6 * max(1.0, abs(r.obj))
        F, u, ok = benders.follower_response(A, B, b, c1, d1, d2, r.p)   # re-solve the follower
        assert ok and np.all(A @ r.p + B @ u <= b + 1e-6)
        assert abs(F - r.obj) <= 1e-6 * max(1.0, abs(r.obj))              # objective is attained


def _synthetic(n, rng):
    df = pd.DataFrame({"r_last": rng.lognormal(-4, 0.3, n), "rho": rng.uniform(0.001, 0.01, n)})
    for i in range(1, 11):
        df[f"r{i}"] = df.r_last * (11 - i)
    for k, lim in QUANTS.items():
        true = rng.normal(lim, 0.05 * (lim if lim > 0.1 else 1), n)
        df[f"{k}_true"] = true
        df[f"{k}_pred"] = true + rng.normal(0, 1, n) * df.r_last * 3
```

```
    df["qex_pred"] = 0.0
    df["slack_pred"] = 0.0
    return df


def test_conformal_miss_rate_is_controlled():
    rng = np.random.default_rng(0)
    misses = []
    for rep in range(30):
        tune, cal, test = (_synthetic(n, rng) for n in (500, 500, 2000))
        for method in ("plain", "normalised", "ptr"):
            S = Screen(alpha=0.1, method=method).fit(tune, cal, seed=rep)
            misses.append(S.report(test)["miss_rate"])
    assert np.mean(misses) <= 0.1 + 0.02
```

tests/test_theory.py

```
"""Numerical checks of the statements proved in the paper's appendix
(run: pytest -q tests). Each test names the result it checks."""
import os

import numpy as np
import pandas as pd
import pytest

from genco_rss import benders, rss
from genco_rss.conformal import _thr
from genco_rss.grid import Grid

RES = os.path.join(os.path.dirname(__file__), "..", "results")
G = Grid.load()
HOURS = [4, 12, 21]


def _limits(h):
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(G, h)
    c1 = np.zeros(A.shape[1])
    up = benders.solve(A, B, b, c1, -d, d, p_lb, p_ub, u_ub)
    lo = benders.solve(A, B, b, c1, d, -d, p_lb, p_ub, u_ub)
    return (A, B, b, d, p_lb, p_ub, u_ub), -up.obj, lo.obj


def _phi_psi(A, B, b, d, p):
    c1 = np.zeros(A.shape[1])
    Fu, _, ok1 = benders.follower_response(A, B, b, c1, -d, d, p)
    Fl, _, ok2 = benders.follower_response(A, B, b, c1, d, -d, p)
    assert ok1 and ok2                                      # Assumption 1
    return -Fu, Fl                                          # phi(p), psi(p)


@pytest.mark.parametrize("h", HOURS)
def test_theorem_rss_interval_and_vertex_lemma(h):
    """Theorem (meaning of the RSS interval) and Lemma (worst cases are vertices):
    for every p in the box (vertices AND interior points), psi(p) <= alpha_min and
    phi(p) >= alpha_max, so every alpha in [alpha_min, alpha_max] is achievable."""
    (A, B, b, d, p_lb, p_ub, u_ub), a_max, a_min = _limits(h)
    rng = np.random.default_rng(h)
    for t in range(40):
        if t % 2:
            p = np.where(rng.random(len(p_lb)) < 0.5, p_lb, p_ub)        # vertex
        else:
            p = p_lb + rng.random(len(p_lb)) * (p_ub - p_lb)            # interior point
        phi, psi = _phi_psi(A, B, b, d, p)
        assert phi >= a_max - 1e-6 and psi <= a_min + 1e-6


@pytest.mark.parametrize("h", HOURS)
def test_lemma_robust_value_function_bound(h):
    """Lemma (robust value-function bound): max_u d2'u >= phi_R for every p."""
    A, B, b, d, p_lb, p_ub, u_lb, u_ub = rss.problem(G, h)
    for d2 in (d, -d):
        phi_R, _ = benders.robust_bound(A, B, b, d2, p_lb, p_ub, u_ub)
```

```
        if not np.isfinite(phi_R):
            continue
        rng = np.random.default_rng(1)
        for _ in range(30):
            p = p_lb + rng.random(len(p_lb)) * (p_ub - p_lb)
            F, _, ok = benders.follower_response(A, B, b, np.zeros(A.shape[1]), d2, d2, p)
            assert ok and F >= phi_R - 1e-6


@pytest.mark.parametrize("h", [12])
def test_concavity_of_phi(h):
    """Lemma (worst cases are vertices): phi is concave and psi convex along segments."""
    (A, B, b, d, p_lb, p_ub, u_ub), _, _ = _limits(h)
    rng = np.random.default_rng(3)
    for _ in range(10):
        p1 = p_lb + rng.random(len(p_lb)) * (p_ub - p_lb)
        p2 = p_lb + rng.random(len(p_lb)) * (p_ub - p_lb)
        th = rng.random()
        f1, g1 = _phi_psi(A, B, b, d, p1)
        f2, g2 = _phi_psi(A, B, b, d, p2)
        ft, gt = _phi_psi(A, B, b, d, th * p1 + (1 - th) * p2)
        assert ft >= th * f1 + (1 - th) * f2 - 1e-6
        assert gt <= th * g1 + (1 - th) * g2 + 1e-6


def test_nesting_of_ac_corrected_limits():
    """Proposition (nesting): AC-corrected limits lie inside the DC limits."""
    e4 = pd.read_csv(os.path.join(RES, "e4_ac_rss.csv"))
    up, lo = e4[e4.limit == "upper"], e4[e4.limit == "lower"]
    assert np.all(up.alpha_ac <= up.alpha_dc + 1e-6)
    assert np.all(lo.alpha_ac >= lo.alpha_dc - 1e-6)


def test_batteries_only_widen():
    """Proposition (batteries can only widen the RSS)."""
    w = pd.read_csv(os.path.join(RES, "e5_bess_width.csv"))
    assert np.all(w.gain >= -1e-6)


def test_conformal_quantile_lemma():
    """Lemma (split-conformal quantile): P(S_{n+1} > c) <= a for exchangeable scores,
    checked by simulation with heavy tails and ties."""
    rng = np.random.default_rng(0)
    n, a, reps = 200, 0.05, 4000
    hits = 0
    for _ in range(reps):
        s = np.round(rng.standard_t(2, n + 1), 1)          # heavy tails + ties
        hits += s[-1] > _thr(s[:-1], a)
    assert hits / reps <= a + 3 * np.sqrt(a * (1 - a) / reps)
```

tools/make_pdfs.py

```
"""Convert the Markdown documents and the source code to PDF
(pandoc -> HTML -> headless Chromium). Usage: python tools/make_pdfs.py OUT_DIR"""
import asyncio, glob, html, os, subprocess, sys
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import get_lexer_for_filename

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
OUT = os.path.abspath(sys.argv[1] if len(sys.argv) > 1 else os.path.join(ROOT, "pdf"))
os.makedirs(OUT, exist_ok=True)
CSS = """
body{font-family:'DejaVu Sans',Arial,sans-serif;font-size:10.5pt;line-height:1.45;color:#1d1d1f;max-
width:none;margin:0}
h1{font-size:19pt;margin:0 0 4pt}h2{font-size:14pt;border-bottom:1px solid #ddd;padding-bottom:3pt;margin-
top:16pt}
h3{font-size:12pt;color:#444}code{font-family:'DejaVu Sans Mono',monospace;font-
size:8.8pt;background:#f4f4f4;padding:0 2px}
pre{background:#f6f6f6;border:1px solid #e3e3e3;padding:6pt;font-size:8pt;white-space:pre-wrap;word-
break:break-word}
pre code{background:none}table{border-collapse:collapse;margin:6pt 0;font-size:9.5pt}
```

```
th,td{border:1px solid #ccc;padding:3pt 6pt}th{background:#f0f3f8}img{max-width:100%;margin:6pt 0}
hr{border:0;border-top:1px solid #ddd}
"""


KATEX = os.environ.get("KATEX_DIR")      # e.g. node_modules/katex/dist (npm install katex) for best math


def md_html(path):
    math = [f"--katex=file://{os.path.abspath(KATEX)}/"] if KATEX else ["--mathml"]
    body = subprocess.run(["pandoc", path, "-f", "gfm+tex_math_dollars-yaml_metadata_block", "-t", "html5",
                           "--standalone", "--metadata", "pagetitle=doc"] + math,
                          capture_output=True, text=True, check=True).stdout
    if KATEX:                                    # keep pandoc's head (KaTeX scripts), add our CSS
        return body.replace("</head>", f"<style>{CSS}</style></head>")
    if body.lstrip().startswith("<hr") and os.path.basename(path) == "README.md":   # HF front matter
        pass
    return f"<html><head><meta charset='utf-8'><style>{CSS}</style></head><body>{body}</body></html>"


def code_html(paths):
    fmt = HtmlFormatter(style="friendly", noclasses=False)
    parts = [f"<h1>Source code</h1><p>{len(paths)} files</p>"]
    for p in paths:
        rel = os.path.relpath(p, ROOT)
        try:
            lex = get_lexer_for_filename(p)
        except Exception:
            from pygments.lexers import TextLexer; lex = TextLexer()
        parts.append(f"<h2>{html.escape(rel)}</h2>" + highlight(open(p).read(), lex, fmt))
    css = CSS + fmt.get_style_defs(".highlight") + ".highlight pre{font-size:7.6pt;line-height:1.3}"
    return f"<html><head><meta charset='utf-8'><style>{css}</style></head><body>{''.join(parts)}</body>
</html>"


async def render(jobs):
    from playwright.async_api import async_playwright
    async with async_playwright() as pw:
        b = await pw.chromium.launch()
        pg = await b.new_page()
        for html_text, base, out in jobs:
            tmp = os.path.join(base, ".__tmp_render.html")
            open(tmp, "w").write(html_text)
            await pg.goto("file://" + tmp)
            await pg.wait_for_load_state("networkidle")
            await pg.pdf(path=out, format="A4", print_background=True,
                         margin={"top": "16mm", "bottom": "16mm", "left": "15mm", "right": "15mm"})
            os.remove(tmp)
            print("wrote", out)
        await b.close()


jobs = []
for md in ["README.md"] + sorted(glob.glob(os.path.join(ROOT, "docs", "*.md"))):
    md = os.path.join(ROOT, md) if not os.path.isabs(md) else md
    src = open(md).read()
    if src.startswith("---"):                    # strip HF front matter for the PDF
        src = src.split("---", 2)[2]
    tmp_md = md + ".tmp.md"; open(tmp_md, "w").write(src)
    jobs.append((md_html(tmp_md), ROOT, os.path.join(OUT, os.path.splitext(os.path.basename(md))[0] +
".pdf")))
    os.remove(tmp_md)
code = [p for pat in ("*.py", "genco_rss/*.py", "experiments/*.py", "tests/*.py", "tools/*.py", "*.sh",
                      "configs/*.yaml", "requirements.txt") for p in sorted(glob.glob(os.path.join(ROOT,
pat)))]
jobs.append((code_html(code), ROOT, os.path.join(OUT, "source_code.pdf")))
asyncio.run(render(jobs))
```

setup.sh

```
#!/usr/bin/env bash
# Environment for "Certified AC robust secure sets with GENCO in the loop".
```

```
# Tested on Linux, Python 3.12, CPU only (gridfm-graphkit 0.9.0.3, gridfm-datakit 1.1.0).
set -euo pipefail
python3.12 -m venv venv
source venv/bin/activate
pip install --upgrade pip

# gridfm-graphkit pins torch >=2.10,<2.13; torch-scatter must match the torch build
pip install "torch==2.12.*" --index-url https://download.pytorch.org/whl/cpu
TORCH_VERSION=$(python -c "import torch; print(torch.__version__.split('+')[0])")
pip install torch-scatter -f "https://data.pyg.org/whl/torch-${TORCH_VERSION}+cpu.html" \
  || pip install torch-scatter --no-build-isolation          # build from source if no wheel

# OpenGridFM toolkits + AC power-flow / OPF back-ends + app
pip install -r requirements-full.txt

# gridfm-graphkit logs to an MLflow file store
export MLFLOW_ALLOW_FILE_STORE=true

# Optional: official GENCO power-flow checkpoint for case118 instead of training
# pip install "huggingface_hub[cli]"
# hf download gridfm/genco-pf-datakit --local-dir genco-pf
# python genco_ac_rssr_challenge.py --steps e3,e4,fig,numbers,docs \
#        --genco-run <folder of the case118 checkpoint> --genco-config <its training yaml>

# Re-run all experiments with the shipped trained GENCO (weights/genco_pf_case118, about 45 min)
# python genco_ac_rssr_challenge.py --steps e1,e3,e4,e5,fig,numbers,docs

# Full reproduction incl. data and training (CPU: about 6 h)
# python genco_ac_rssr_challenge.py
```

configs/genco_pf_case118.yaml

```
# GENCO (GNS_heterogeneous) power-flow model for PGLib case118, small configuration
# trained from scratch on CPU with gridfm-graphkit (official checkpoints: gridfm/genco-pf-datakit).
version: 1
callbacks:
  patience: 100
  tol: 0
  early_stopping_monitor: Validation loss
  checkpoint_monitor: Validation loss
  lr_scheduler_monitor: Validation loss
task:
  task_name: PowerFlow
data:
  baseMVA: 100
  mask_value: 0.0
  normalization: HeteroDataMVANormalizer
  networks:
  - case118_ieee
  scenarios:
  - 24000
  test_ratio: 0.1
  val_ratio: 0.1
  workers: 0
  split_by_load_scenario_idx: true
  stream_partitions: auto
model:
  attention_head: 4
  edge_dim: 10
  hidden_size: 24
  input_bus_dim: 15
  input_gen_dim: 6
  output_bus_dim: 2
  output_gen_dim: 1
  num_layers: 10
  type: GNS_heterogeneous
optimizer:
  type: AdamW
  learning_rate: 0.0005
  optimizer_params:
    betas: [0.9, 0.999]
  scheduler_type:  ReduceLROnPlateau
```

```yaml
  scheduler_params:
    mode: min
    factor: 0.7      # previously lr_decay
    patience: 5      # previously lr_patience
seed: 0
training:
  batch_size: 64
  epochs: 10
  loss_weights:
  - 0.1
  - 0.9
  losses:
  - LayeredWeightedPhysics
  - MaskedBusMSE
  loss_args:
  - base_weight: 0.5
  - {}
  physics_warmup_epochs: 0 # set >0 to ramp LayeredWeightedPhysics in linearly
  accelerator: auto
  devices: auto
  strategy: auto
verbose: true
```

requirements.txt

```
# Hugging Face Space (app.py): results viewer, live Benders RSS, live conformal screen.
# The full pipeline (data generation, GENCO training, exact AC) uses requirements-full.txt via setup.sh.
scipy>=1.14
numpy>=2.0
pandas>=2.2
pyarrow>=15
scikit-learn>=1.5
matplotlib>=3.8
tabulate
```